Master's Thesis

Escaping the Plasticity-Memory Dilemma in Intelligent Robotic Systems

Faculty of Physics
Ludwig-Maximilians-Universität München

Rajat Saxena

Munich, 24 September, 2026

Submitted in partial fulfilment of the requirements for the degree of M. Sc. Physics

Supervised by Prof Steffen Rulands

Masterarbeit

Überwindung des Dilemmas zwischen Plastizität und Gedächtnis in intelligenten Robotersystemen

Fakultät für Physik
Ludwig-Maximilians-Universität München

Rajat Saxena

München, 24 September, 2026

Eingereicht zur teilweisen Erfüllung der Voraussetzungen für den Abschluss M. Sc. Physik

Betreut von Prof Steffen Rulands

Abstract

With the advent of artificial intelligence, machine learning has become more widely applied in robotics. In experiments such as foraging, neural networks can learn to identify and distinguish between multiple resources for collection and deposition. However, a persistent challenge in learning systems is the stability-plasticity dilemma: learning systems must remain plastic enough to learn new tasks while retaining information about previously learned functions. This trade-off can also arise in intelligent robots, where learning to classify and forage for one resource can lead to the loss of information acquired from previous tasks, a phenomenon known as catastrophic forgetting. One approach to addressing this problem is to use multi-robot or swarm systems, in which multiple interacting robots perform tasks together, enabling decentralised learning. However, such systems typically rely on a coordinator or controller to allocate tasks to different robots and reassign them when required. Specialisation and the resulting distribution of memory and plasticity therefore depend on centralised task allocation. In this thesis, a performance-dependent suppression interaction that gates training opportunities is introduced for an ensemble of interacting and competing robots.

This thesis implements a minimal experimental setup using mobile robots with different sensor modalities for foraging experiments. The robots' trajectories are tracked alongside network performance on different resource-identification tasks to determine whether specialisation emerges. The results show that self-organised specialisation can emerge in the robot ensemble without explicit task assignment, with individual agents developing preferences for particular resources.

The thesis also introduces an active-matter model to study the mechanisms underlying self-organised specialisation.

Acknowledgements

I would firstly like to express my sincere gratitude to Prof. Steffen Rulands for his guidance and support. Every meeting with Prof. Rulands not only motivated me to pursue the project but also reminded me why I chose a career in Physics. I would also like to thank him for allowing me to convert his room into a robotics lab and for providing the resources I needed for the experiments. I want to thank Onurcan Bektas for all the meetings, brainstorming sessions, and for letting me temporarily convert his office into a mini-robotics lab. Thank you both for being such great guides throughout this thesis. I would also like to thank Severin Angerpointner for promptly ordering the robots and other requirements.

I truly value the enriching discussions with Prof. Thomas Speck and his group on active matter systems with internal degrees of freedom, which led to the entire Chapter 5 of my thesis. I would also like to thank Gerhard Woithe and Nikolas Pfaffenzeller for the discussions on Physics and beyond.

I dedicate this thesis, and the nearly two years of effort I spent in Munich, to my parents, without whom this endeavour wouldn't have been possible. I want to thank them for all their efforts and for being a part of my happiness.

Finally, I would like to thank my friends here in Munich, especially Xi-vesh Dwivedi (Shivesh), Yash Raj Sood, the Badminton group, and my friends from the Associazione Italiana Studenti di Fisica. I thank Yash and Xi-vesh for also helping me build the experimental arena.

Contents

1 Introduction **1**

2 Theoretical Background **3**
2.1 Neural Network Training for Classification Tasks 3
2.2 Stochastic Processes and the Gillespie Algorithm 5
2.2.1 Deterministic and Stochastic Dynamics 5
2.2.2 Continuous-Time Markov Jump Processes 5
2.2.3 Gillespie Simulation Algorithm 6
2.2.4 Poisson Thinning for State-Dependent Event Rates 6
2.2.5 Tau-Leaping for Collective Events 7
2.2.6 State-Dependent Markov Networks and Generators 7
2.2.7 Coupled Switching Diffusion Processes 7
2.3 Active Matter and Run-and-Tumble Dynamics 8

3 Introduction to the Experimental Setup **9**
3.1 Robot Experimental Platform 9
3.2 Hardware Components 10
3.2.1 ESP32 10
3.2.2 Arduino UNO 12
3.2.3 Ultrasonic Sensor 12
3.2.4 IR 12
3.2.5 MPU600 13
3.3 Experimental Design with Given Constraints 13
3.3.1 The Ultrasound Sensor 13
3.3.2 IR Sensor Array 14
3.3.3 Tracking with Camera 15
3.4 Communication Protocols and Information Flow 15
3.4.1 WiFI and UDP Handshake 17
3.4.2 Serial Communication via UART 17
3.5 Introduction to State Machine 18
3.6 Reverse Central Foraging Problem 18
3.7 Experimental Layout 19
3.7.1 AprilTag as Task Markers 19
3.7.2 Central Blue Cylinder 20
3.7.3 White Hexagon and Tape Markings 20

3.8 Computational Pipeline . . . 21
3.8.1 Arduino and ESP32 code . . . 22
3.8.2 State Machine Architecture for Detection and Navigation . . . 23
3.8.3 Robot Tracking and Trajectory Calculation . . . 26

4 Results **28**
4.1 Numerical Realisation of the Decentralised Learning Ensemble . . . 28
4.1.1 The Collective Solution to the Stability-Plasticity Dilemma . . . 28
4.1.2 System Architecture and State Representation . . . 28
4.1.3 Synthetic Gaussian Training Data and Task Orthogonality . . . 29
4.1.4 Stochastic Event-Driven Gating Dynamics . . . 29
4.1.5 Simulation Results . . . 30
4.1.6 Characterization of Phase Space and Dynamical Regimes . . . 30
4.1.7 Ensemble Plasticity and Reconfigurability . . . 32
4.2 Self-organisised Specialisation in Robotic Ensembles . . . 35
4.2.1 Environment-Dependent Event Gating in Physical Multi-Agent Systems . . . 35
4.2.2 Training data . . . 37
4.2.3 Foraging Event Sequence . . . 38
4.2.4 τ-leaping based event-driven dynamics . . . 39
4.2.5 Results for an Ensemble of Six Robots and Six Tasks . . . 41

5 Active-Matter Model of Collective Specialisation **44**
5.1 Limitations of the Neural-Agent Ensemble . . . 44
5.2 Introduction to an Active-Matter Description . . . 45
5.3 Model Definition and Numerical Implementation . . . 46
5.3.1 Performance-Space Dynamics . . . 46
5.3.2 Internal-State Network . . . 47
5.3.3 Numerical Integration of the Continuous Dynamics . . . 49
5.3.4 State-Dependent Switching by Approximate Thinning . . . 49
5.3.5 Time-Discretised Event Evaluation . . . 50
5.3.6 Simulation Algorithm . . . 51
5.4 Collective Behaviour and Reproduction of Neural-Ensemble Phenomena . . . 51
5.4.1 Emergence of Stable Specialisation . . . 51
5.4.2 Collective Phase Diagram . . . 52
5.4.3 Scaling with System Size . . . 53
5.4.4 Plasticity and Memory . . . 53
5.4.5 Role of Harmonic Relaxation . . . 55
5.4.6 Role of Diffusive Fluctuations . . . 56
5.4.7 Role of Inactive-Task Decay and Task Orthogonality . . . 56
5.5 Summary, Comparison, and Outlook . . . 57

6 Discusssion **58**

Bibliography **60**

Chapter 1

Introduction

Machine learning has become an important component of modern robotic systems, allowing robots to acquire behaviours from data and interaction with their environment rather than relying entirely on manually specified rules or predefined instructions. Learning-based approaches have been applied to perception, decision-making, control, and skill acquisition, enabling robots to adapt their behaviour based on observations of their environment [1]. This is particularly relevant for autonomous robots operating in changing environments, where a robot may need to incorporate new information, adapt to changes in its surroundings, and retain knowledge acquired from previous experiences.

Such lifelong learning problems fall under continual learning, in which an agent receives information sequentially and is expected to learn new tasks while retaining previously acquired knowledge [1, 2]. This setting is particularly relevant to robotic foraging experiments, where a robot explores an arena and encounters different resources during its operation. As the robot encounters new resources, it must learn to identify them while retaining the ability to identify resources it has encountered previously.

A central problem in continual learning is the trade-off between plasticity and memory, often called the stability–plasticity dilemma. An "intelligent robot: or a robot endowed with a neural network, requires sufficient plasticity to incorporate new information, but changes made while learning can interfere with previously acquired knowledge. Conversely, mechanisms that strongly preserve existing knowledge can limit the system's ability to adapt to new information [2]. In neural networks, this interference can manifest as catastrophic forgetting, in which learning subsequent tasks degrades performance on previously learned tasks [2]. In a robotic foraging setting, for example, a robot that learns to identify a new resource may consequently reduce its ability to identify a resource learned earlier.

One way of resolving this problem is to use multiple robots rather than requiring a single robot to perform all foraging tasks. Available tasks can be distributed among a group of robots, allowing individual robots or subgroups to focus on different parts of the problem. This idea is closely related to task allocation and division of labour observed in biological collectives [3, 4]. In swarm robotics, foraging is a commonly studied task in which collective behaviour can arise through task allocation by a centralised controller or through

predefined rules for the robots [3–5]. By distributing tasks across several robots, learning becomes decentralised, while task-solving and resource-collection abilities remain at the system level.

A centralised controller uses the information about the robots and available tasks to determine allocation [3]. Specialisation is therefore introduced through an explicit allocation mechanism or through rules designed to produce a desired division of labour. This raises the question of whether specialisation can emerge from the system's learning dynamics, without explicitly assigning roles to individual robots.

In this thesis, we introduce a self-organised method for obtaining specialisation in a system of interacting robots. The premise of the experiment is that each foraging event for a particular resource improves the robot's ability to identify that resource, since each foraging event trains the robot's neural network. We implement suppression-driven dynamics in which, when two agents compete for the same task, one agent is suppressed from performing that task for a fixed period [6]. During this period, the suppressed agent does not forage for that resource and remains active on others. Pfaffenzeller [6] showed that, for an appropriate choice of hyperparameters, this gated training can lead to specialisation. This was shown for a system of competing neural networks.

In this thesis, we investigate whether the same underlying mechanism can be implemented in a system of mobile robots performing a foraging experiment, and whether specialisation can emerge under constraints imposed by physical conditions. To investigate this, we use an ensemble of mobile robots equipped with sensors to observe the environment and respond to task-related stimuli.

This thesis does not aim to reproduce the neural network ensemble results of Pfaffenzeller [6] in physical form. Instead, we develop a minimal robotic system to test the proposed suppression-driven dynamics in a standard foraging experiment under physical constraints. These constraints include the time required for a robot to reach a target, failures to reach the target, and suppression's effects on the availability of subsequent foraging events. The key questions addressed in this thesis are: (1) Can suppression-driven dynamics lead to stable specialisation in a robotic foraging system? (2) What choice of hyperparameters leads to specialisation under strict time constraints, and what fraction of resources are assigned to a unique robot? (3) If stable specialisation is achieved, can the collective remain sufficiently plastic to respecialize when a perturbation, such as the addition of a new resource, is introduced?

To study decentralised learning and self-organised specialisation in multi-agent systems, an active matter model is introduced. The model coarse-grains neural-network learning dynamics into agent-level continuous dynamics and stochastic state transitions, replacing the computationally expensive global simulation with an agent-level description. This provides a tractable framework for studying specialisation over longer time scales while retaining the essential interactions between learning, suppression, and training events.

Chapter 2

Theoretical Background

We first introduce the theoretical concepts used throughout the thesis. These include neural networks for classification tasks, stochastic processes and simulations, and active matter with internal degrees of freedom.

2.1 Neural Network Training for Classification Tasks

A fully connected feedforward neural network (FFNN) is an architecture in which the neurons of one layer are connected to the neurons of the following layer. Information therefore propagates from the input layer to the output layer [7]. In supervised learning, the network represents a parameterised mapping from the input data to the output class labels. The parameters consist of weights and biases that are adjusted during training to minimise a defined loss function [7].

The input layer has a dimensionality determined by the input data, followed by layers with chosen dimensions that form the hidden layers. The trainable weights determine the strength of the connections between neurons in successive layers. The hidden layers transform the input representation before the final output layer, whose dimensionality is equal to the number of output classes.

To allow the multilayer architecture to represent non-linear relationships in the data, an element-wise activation function is applied to the output of each neuron. The bias acts as an adjustable offset, allowing the activation of each neuron to shift independently of the input. Without an activation function, a multilayer network remains equivalent to a linear transformation, so increasing its depth does not introduce additional non-linearity.

Throughout the thesis, the Rectified Linear Unit (ReLU) was used as the activation function [8], which is defined as:

$$\mathrm{ReLU}(x) = \begin{cases} 0, & x < 0, \\ x, & x \geq 0. \end{cases} \tag{2.1.1}$$

A *forward pass* processes the input data through these weights, biases, and activation functions to produce the output at the output layer. The unnormalised outputs, or logits,

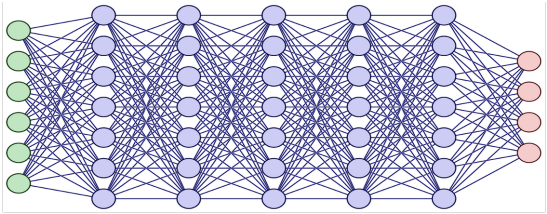


Figure 2.1.1: A dense feed-forward neural network architecture, illustrating feedforward signal propagation from the input layer to the output layer.

produced from the output layer are then mapped to a valid probability distribution using the Softmax function [7], given as

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{C} e^{z_j}}, \quad i = 1, 2, \ldots, C. \tag{2.1.2}$$

Applying the softmax function gives values between 0 and 1 that sum to one, and the predicted class is selected using the argmax of these probabilities.

For evaluation and backpropagation, we use categorical cross-entropy loss. It measures the difference between the target class distribution and the distribution predicted by the model [7]. For a target vector $\mathbf{y}$ and predicted vector $\hat{\mathbf{y}}$, the categorical cross-entropy loss is given by

$$\mathcal{L}_{\text{CCE}} = -\sum_{i=1}^{C} y_i \log(\hat{y}_i). \tag{2.1.3}$$

Minimising this loss penalises incorrect predictions and increases the contribution of the correct target to the loss. Training is performed iteratively by updating the network parameters to reduce the loss. For a parameter vector $\mathbf{w}$, a standard gradient-descent update is

$$\mathbf{w}' = \mathbf{w} - \gamma \nabla_{\mathbf{w}} \mathcal{L}, \tag{2.1.4}$$

where $\gamma > 0$ is the learning rate [7].

Backpropagation computes the required parameter gradients by applying the chain rule through the network in reverse order [9]. Several optimisation methods extend stochastic gradient descent by adapting the parameter updates. Throughout the thesis, we use the Adam optimiser, which scales parameter updates using estimates of the first and second moments of the gradients [10]. The training process can therefore be viewed as an optimisation in a high-dimensional parameter space, with successive parameter updates determining the evolution of the network.

In addition to the loss computed during training, other observables can be used to measure network performance. One such metric is classification accuracy, which gives the fraction of labels predicted correctly. For a dataset of size N, it is given as:

$$a = \frac{1}{N} \sum_{i=1}^{N} \mathbb{I}(\hat{y}_i = y_i). \tag{2.1.5}$$

where $\mathbb{I}(\cdot)$ denotes the indicator function, which is 1 when its condition is satisfied and 0 otherwise.

2.2 Stochastic Processes and the Gillespie Algorithm

This section introduces continuous-time Markov jump processes, the Stochastic Simulation Algorithm, and Markov networks used in the thesis.

2.2.1 Deterministic and Stochastic Dynamics

Deterministic dynamical systems have trajectories that are uniquely determined by their initial conditions and the corresponding ordinary or partial differential equations. This description is appropriate when the relevant degrees of freedom can be explicitly tracked or resolved [11, 12].

For complex systems composed of many interacting particles, a complete deterministic description can become impractical when microscopic states or internal degrees of freedom are inaccessible or coarse-grained. In such cases, the time evolution is described probabilistically [11]. Stochastic models represent this evolution using probabilistic transition rules and noise [11].

Stochasticity at the system level does not imply that the ensemble lacks regular structure. Individual realisations can exhibit macroscopic properties such as stationary distributions and collective behaviour [12]. In this thesis, we consider systems with discrete and asynchronous state transitions in continuous time, which can be described by continuous-time jump processes.

2.2.2 Continuous-Time Markov Jump Processes

A stochastic process is a collection of random variables $\{X(t)\}_{t\geq 0}$ taking values in a discrete state space $\mathcal{S}$. A sample trajectory is one realisation of the stochastic process from an initial state through the state space [11]. A continuous-time Markov jump process (CTMJP) is a subclass in which the conditional distribution of the future state depends only on the current state. This can be written as

$$P(X(t) = x \mid X(s) = x_s, X(u) = x_u) = P(X(t) = x \mid X(s) = x_s), \tag{2.2.1}$$

for any sequence of times $t > s > u$.
In a CTMJP, a system remains in a state for a random or fixed time before transitioning to another state. The transition rates and the time spent in each state therefore determine the evolution of the system. The rates may be fixed or may depend on the current system state. These transition rates define the master equation, which describes the evolution of

the probability distribution over the state space [12]. For complex systems, the state space can be too large for an analytical solution. Individual trajectories can instead be generated such that their ensemble properties reproduce the corresponding stochastic process [12]. The Gillespie algorithm provides an exact sampling procedure for this purpose [13].

2.2.3 Gillespie Simulation Algorithm

The Gillespie Stochastic Simulation Algorithm (SSA) generates an exact realisation of a CTMJP by sampling successive events and their occurrence times [13]. At state x and time t, each permissible event channel j is assigned a state-dependent propensity $a_j(x)$. The total event rate is

$$a_0(x) = \sum_j a_j(x). \tag{2.2.2}$$

The waiting time until the next event is then sampled from an exponential distribution with rate $a_0(x)$,

$$\Delta t = \frac{1}{a_0(x)} \ln\left(\frac{1}{r_1}\right), \qquad r_1 \sim \mathcal{U}(0,1), \tag{2.2.3}$$

and a candidate event is selected with probability $a_j(x)/a_0(x)$ using a second independent random number $r_2 \sim \mathcal{U}(0,1)$. The system is then updated according to the selected event, $x \to x^{(j)}$, and the simulation time advances by Δt. Repeating the procedure generates a trajectory of the continuous-time stochastic process without a fixed time step.

2.2.4 Poisson Thinning for State-Dependent Event Rates

The Gillespie algorithm introduced in Section 2.2.3 assumes that the system state remains unchanged during each waiting interval, so the propensities also remain constant [13]. This assumption does not hold for the active matter model discussed in Chapter 5. There, the transition rates depend on the continuously evolving particle positions $X(t)$, which change according to Langevin dynamics. The transition intensities therefore vary during the waiting interval, so the standard Gillespie algorithm cannot be applied directly.
The thinning algorithm provides an alternative for generating stochastic events when transition rates depend on time or on an evolving state [14]. The method introduces a homogeneous Poisson process with a constant dominating rate $\bar{\lambda}$ satisfying

$$\lambda(t) \leq \bar{\lambda} \tag{2.2.4}$$

We first generate candidate events from this dominating rate. Since $\bar{\lambda}$ is constant, the waiting time between candidate events is exponentially distributed,

$$\Delta t_{\text{cand}} = -\frac{\ln r_1}{\bar{\lambda}}, \qquad r_1 \sim \mathcal{U}(0,1). \tag{2.2.5}$$

At the candidate time t_{cand}, the instantaneous transition rate $\lambda(t_{\text{cand}})$ is evaluated. The candidate event is then accepted with probability

$$P_{\text{acc}} = \frac{\lambda(t_{\text{cand}})}{\bar{\lambda}}, \tag{2.2.6}$$

and rejected otherwise. Repeating this procedure removes candidate events that are not accepted, leaving events distributed according to the desired non-homogeneous intensity [14].

2.2.5 Tau-Leaping for Collective Events

The Gillespie algorithm schedules and executes events sequentially. At each step, one event is selected, the system is updated, and the simulation time advances. For systems in which multiple interacting particles must act on similar timescales, this sequential procedure can become a computational bottleneck. The τ-leaping method instead considers a finite interval τ during which the event rates are assumed to remain approximately constant, allowing events sampled over the interval to be executed together [15]. The number of occurrences of event j during an interval can be sampled from a Poisson distribution as

$$K_j(\tau) \sim \text{Poisson}\left[a_j(x)\tau\right]. \tag{2.2.7}$$

After completing the events scheduled for that interval, the system can advance directly from t to $t+\tau$. Repeating this process over all τ intervals generates the system's stochastic trajectory. This is useful in systems where the overall effect of events matters more than their individual ordering.

The approximation requires τ to be small enough that the event rates do not change substantially within the interval. The choice of τ therefore affects both the accuracy of the approximation and the number of events that can be processed together. Smaller values more closely reproduce the sequential Gillespie dynamics, while larger values allow more events to be processed collectively [15, 16].

2.2.6 State-Dependent Markov Networks and Generators

A discrete internal state space of particles in a system can be mapped to a Markov Network, where each vertex represents an internal state and edges define the allowed transition pathways. For example, each particle has a directed graph represented by $G = (V, E)$, where the vertices $i \in V$ correspond to explicit states, and directed edges $(i \to j) \in E$ define permissible pathways through which a transition from state i to j occurs [17].

Markov networks can also have state-dependent transition rates, allowing the system configuration to affect the internal dynamics of the particles. In this case, the transition rates are not fixed and can change with the system configuration and feedback between the internal states and the surrounding system [18, 19].

2.2.7 Coupled Switching Diffusion Processes

Switching diffusion processes combine continuous stochastic motion with discrete Markovian switching. A continuous variable $X(t)$ evolves according to a stochastic differential equation whose drift or diffusion coefficients depend on a discrete internal state $M(t)$,

which can be represented by a Markov network. The system configuration and $X(t)$ can also determine the transition rates of the network. A general switching diffusion may be written as

$$dX(t) = b(X(t), M(t))dt + \sigma(X(t), M(t))dW(t), \tag{2.2.8}$$

where $W(t)$ is a Wiener process. Simultaneously, the discrete internal state undergoes transitions

$$M : m \to n \tag{2.2.9}$$

with instantaneous rates

$$q_{mn}(X). \tag{2.2.10}$$

The internal state modifies the continuous dynamics through $b(X, M)$, while the continuous coordinates affect the internal-state dynamics through the state-dependent transition rates. The two components are therefore coupled in both directions.

2.3 Active Matter and Run-and-Tumble Dynamics

Active matter systems consist of individual units that continuously consume energy and convert it into directed motion, mechanical force or internal state transitions. Active particles have an internal source of propulsion, along with thermal fluctuations and externally applied forces that drive the system out of equilibrium. Examples range from motile microorganisms and biological cells to Janus spheres [20, 21]. Their dynamics can be described using stochastic equations with self-propulsion in the Langevin Equations.

Run-and-tumble particles were originally inspired by the motion of motile bacteria such as *Escherichia coli*, whose trajectories consist of persistent runs interrupted by stochastic tumbling events that reorient the direction of motion [22]. In the simplest case, a particle moves with approximately constant propulsion velocity during a run,

$$\dot{\mathbf{x}}(t) = v_0 \mathbf{n}(t), \tag{2.3.1}$$

while random tumble events modify $\mathbf{n}(t)$. A run-and-tumble model can be coupled to a switching diffusion process by introducing an internal state $M(t)$ [23]. For example, in one spatial dimension, the two states

$$M(t) \in \{+, -\} \tag{2.3.2}$$

may represent propulsion to the right and left directions, respectively, such that

$$\dot{x} = v(M) + \sqrt{2D}\,\xi(t), \tag{2.3.3}$$

with

$$v(+) = v_0, \qquad v(-) = -v_0. \tag{2.3.4}$$

Here, a transition between the two states reverses the direction of propulsion. The particle's trajectory in physical space is determined by its current internal state and the time spent in that state. The two-state model can be generalised to particles with multiple internal states and motion in multiple dimensions.

Chapter 3

Introduction to the Experimental Setup

This chapter describes the physical experimental platform, its hardware and communication architecture, and the control protocols used for navigation. It also describes the constraints imposed by the available sensory modalities and their effect on the design of the physical experiments.

3.1 Robot Experimental Platform

The experimental platform is the ELEGOO Smart Robot Car Kit V4.0 [24]. The platform is a compact differential-drive mobile robot equipped with an Arduino UNO, an ultrasonic distance sensor, an array of infrared line-tracking sensors, an inertial measurement unit and an ESP32 Camera module capable of WiFi and Bluetooth communication. The ESP32 Camera module and the ultrasonic sensor are mounted on a servo motor, and it also includes RGB illumination and an IR receiver module for remote controls. The official firmware also supports voltage monitoring and several autonomous operating modes [24, 25]. Fig. 3.1.1 shows the robot car used for the experiment.

In the experiments presented in this thesis, the robots act as actuators and do not make local decisions. They rely on an external pipeline for instructions because of the limited computational resources available on the robot. The robots provide telemetry and video signals to the decision-making pipeline and execute the movement commands. The high-level navigation logic and video detection run externally on a computer, which processes camera images from individual robots and sends navigational commands.

The Arduino and ESP32 can perform simple detection and actuation locally. These capabilities are not used for local decision-making because locally executed Arduino routines could conflict with pipeline instructions. The pipeline therefore controls the robot behaviour.

Figure 3.1.1: ELEGOO Smart Robot Car V4.0 used as the physical platform for the robotic implementation of the decentralised learning framework. The principal hardware components are indicated. Image Credits: ELEGOO Blog

3.2 Hardware Components

Table 3.1 summarises the principal sensors available on the platform.

Table 3.1: Primary components available on the ELEGOO Smart Robot Car V4.0.

Sensor	**Measured Quantity**	**Primary Function**
ESP32 Camera	RGB images and Wi-Fi Communication	Visual perception
Ultrasonic sensor	Distance	Obstacle detection
Infrared sensors	Surface reflectance	Line following
Servo motor	Camera orientation	Active vision
Accelerometers	Positions and velocity	Motion estimation

3.2.1 ESP32

The ELEGOO robot uses an ESP32-S3-WROOM-1-Camera [24]. The chip includes an OV2640 or OV3660 camera module, a 2.4 GHz internal WiFi antenna, 8 MB of FLASH and 8 MB of external PSRAM [26]. It provides a video stream for identifying and locating task objects within the robot's field of view, which is approximately 66.5°. The camera stream is accessed through an HTTP endpoint. The pipeline has a dedicated frame-reading thread that continuously acquires frames from the stream and stores the most recent frame, so that it uses the latest image rather than processing every frame sequentially. This prevents stale frames from accumulating and reduces delays in decision-making.

The OV2640 and OV3660 camera modules produced noticeably different image quality at their default settings. In particular, the OV3660 exhibited a cloudier video feed and poorer colour accuracy than the OV2640. We attributed this difference to the greater

number of image-processing parameters available for fine-tuning on the OV3660. We selected and applied the camera settings listed in Table 3.2 to obtain comparable video feeds across robots with different camera modules, reduce camera-dependent biases, and improve target detection. We empirically adjusted these settings based on ambient lighting conditions and the experimental environment background.

Table 3.2: Camera configuration used in the robotic vision system for the OV2640 and OV3660 sensors. Parameters listed as "default" are not explicitly modified by the program. These quantities determine the HSV values of the colours, important for detection.

Camera parameter	**OV2640**	**OV3660**
Pixel format	JPEG	JPEG
Resolution	SVGA (800×600)	SVGA (800×600)
JPEG quality	8	8
Number of frame buffers	2	2
Frame acquisition mode	Latest frame	Latest frame
Frame buffer memory	PSRAM	PSRAM
XCLK frequency	20 MHz	20 MHz
Vertical flip	1	1
Horizontal mirror	1	1
Brightness	Default	0
Contrast	Default	0
Saturation	Default	0
Sharpness	Default	0
Raw gamma correction	Default	Enabled
Lens correction	Default	Enabled
White balance	Default	Enabled
AWB gain	Default	Enabled
White-balance mode	Default	Auto (0)
Automatic exposure control	Default	Enabled initially, then disabled
Automatic gain control	Default	Enabled initially, then disabled
AEC2	Default	Disabled
DCW	Default	Disabled
Gain ceiling	Default	1
Exposure settling delay	—	2 s

The ESP32 has 3 WiFi communication modes: Station (STA) mode, Access Point (AP) mode, and AP+STA (Dual/Coexistence) mode. In Station (STA) mode, it connects to an existing Wi-Fi network; in Access Point (AP) mode, it creates its own Wi-Fi network; and in STA+AP mode, it performs both simultaneously. For this thesis, we use STA mode,

with multiple robots connected to the same WiFi network. Each ESP32 has a unique IP address on the network, allowing for easy communication and unique identification. The pipeline transmits the movement commands to the corresponding robot, while the robot returns telemetry data.

3.2.2 Arduino UNO

The Arduino Uno performs all low-level robot operations and is based on the ATmega328P 8-bit AVR microcontroller. The processor operates at a clock frequency of 16 MHz and has 32 kB of Flash memory, 2 kB of SRAM, and 1 kB of EEPROM [27]. It has the digital, analogue, PWM, and serial interfaces required to connect the robot's motors, ESP32 and sensors.

In the experiment, the Arduino UNO periodically reads the three IR tracking sensors, the ultrasonic distance sensor, and the MPU6050, packages the measurements into a JSON-formatted telemetry message, and transmits it to the ESP32 via serial communication at 50 ms intervals and a given baud rate. At the same time, it continuously monitors the serial interface for incoming movement commands. Depending on the received command, the Arduino directly actuates the motors to move forward, turn left or right, stop, perform a scanning spin, or move backwards. The Arduino therefore performs direct sensor acquisition and motor actuation, while the external pipeline interprets these measurements and selects the appropriate command. The Arduino expects the controller to send instructions as integers; therefore, before transmission, the pipeline maps string-formatted instructions to integers.

3.2.3 Ultrasonic Sensor

The robot is equipped with an ultrasonic distance sensor to detect obstacles in front of it. The sensor consists of an emitter and a receiver and measures distance by measuring the propagation time of an emitted ultrasonic pulse and its reflected signal. The sensor is connected to the Arduino through dedicated trigger and echo pins [24]. In the experimental controller, the measured distance is transmitted as part of the robot's telemetry. The high-level controller compares the measured distance against a predefined stopping threshold.

In close encounters or when the robots face each other head-on, acoustic crosstalk can occur because one sensor detects the other robot's ultrasound pulse and misinterprets it as its own signal. This can lead to errors in distance measurements and trigger false positives for object avoidance. When the robot detects an obstacle, it waits for a predefined amount of time to make sure the distance wasn't an instantaneous error due to crosstalk.

3.2.4 IR

The robot contains three infrared line-tracking sensors positioned across its front underside. The sensors primarily identify the boundaries and designated regions of the

experimental arena [24]. Each colour produces a different sensor reading based on the amount of radiation it absorbs and reflects. Therefore, we used distinct colour tapes to mark different regions of the experimental arena. The central controller received three distinct readings, each corresponding to one sensor, as a JSON file. If two of the three readings changed, the robot triggered the action defined for that colour.

3.2.5 MPU600

The robot includes an MPU6050 inertial measurement unit connected to the Arduino through the I^2C interface. The MPU6050 provides inertial measurements that can estimate changes in the robot's orientation and motion. In the present experiments, the IMU is not used for navigation. Instead, the Arduino uses it locally to compensate for differences in motor power between the two drive wheels. This ensures that movement commands such as "Forward" result in approximately straight-line motion despite inherent asymmetries in the individual motors.

When the robot powers on, it must be stationary because, as the sensors initialise, the MPU6050 calibrates this as the initial position and orientation and uses this configuration as the reference for all further readings.

3.3 Experimental Design with Given Constraints

We first consider the capabilities and limitations of the onboard sensors and how they constrain the possible experimental configurations. To keep the experimental design simple, we designate one sensing modality as the primary source of navigational information, while the remaining sensors serve as safety backups. The experiment is structured around classification and foraging tasks that require two distinct regions: a collector region, where agents acquire a resource or data associated with a task, and a deposit region, where they deliver or evaluate the acquired resource or data. The deposit region should have a unique marker allowing robots to differentiate between them. Given these requirements and sensor limitations, we examine the IR sensor array, ultrasonic sensor, and camera as possible primary navigation methods.

3.3.1 The Ultrasound Sensor

According to the HC-SR04 ultrasound sensor datasheet [28], the sensor has a horizontal field of view (FOV) of about 21° and a vertical field of view of about 4°, with an angular resolution of about 0.6°–1.4°. These characteristics limit the spatial features the sensor can reliably detect. Unique task markers detectable by ultrasound must have sufficiently broad horizontal features to remain distinguishable within the sensor's angular resolution and field of view. The limited FOV and angular resolution make markers harder to distinguish at a distance, particularly when the robot approaches them at oblique angles.

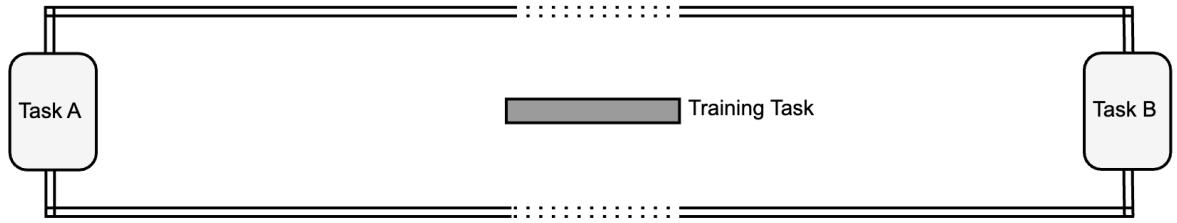


Figure 3.3.1: Schematic of the planned two-agent, two-task experimental environment. Black lines denote the predefined tracks the robots follow to reach Task A and Task B, while the central grey region represents an obstacle. The marked central region indicates where resources are collected.

Because of these limitations, relying only on the ultrasound sensor for navigation and task marker identification is unsuitable for multi-agent experiments. However, it can still be used as an auxiliary sensing modality for collision avoidance. The narrow FOV introduces blind spots, especially when robots approach each other perpendicularly or obliquely.

3.3.2 IR Sensor Array

The IR sensor array allows the robot to detect and follow a line of contrasting colour on the ground. For a small number of agents and tasks, such as two agents performing two tasks, this can support a simple navigation scheme, as illustrated in Fig. 3.3.1. In this configuration, black tape defines fixed tracks connecting the robot's starting region to the two task regions, while the dotted marked region near the centre denotes where the robot stops to collect the required resource. Each direction corresponds to a specific task; therefore, the pipeline only needs to decide whether the robot should follow the left or right track, simplifying the decision-making. In this scheme, the robot can perform the line tracking and other simple navigational computations locally. Allowing, the robot to autonomously follow the corresponding track to the task and return to the starting region, without requiring further navigation instructions from the central controller.

We implemented this experimental configuration as an initial proof of concept. During the experiment, the pipeline used the camera to scan a QR code located at the centre of the arena. The QR code provided the training data the agent needed, which it used to train its neural network and then used the resulting prediction to determine which direction to follow.

Although this approach is straightforward for a small number of agents and tasks, extending it to a larger multi-agent system introduces substantial scalability limitations. Each task requires a corresponding track and direction, so track-topology complexity increases with the number of agents and tasks. Shared tracks also create collision-avoidance challenges. As the robots are confined to the tracks, they have limited freedom to manoeuvre to avoid the other robots. Avoidance would require additional branches in the task layout, further increasing the complexity of the experimental environment.

We therefore used the IR sensor array for detecting environmental boundaries and identifying different areas, while the camera provides the primary navigation information. This removes the need for predefined tracks and allows the same experimental framework to scale to more agents and tasks.

3.3.3 Tracking with Camera

Cameras provide an alternative to line-based navigation because the robot can identify task-specific markers directly in its environment. The camera can continuously observe the environment and identify relevant markers using computer vision. However, running the full computer-vision detector locally on the ESP32 would impose substantial computational constraints, particularly while maintaining a sufficiently high frame rate and image quality and avoiding processing delays. Consequently, the ESP32 acquires the video feed, while the pipeline running on an external computer handles image processing and navigation decisions. This increases the computational requirements of the pipeline but gives greater flexibility in visual perception.

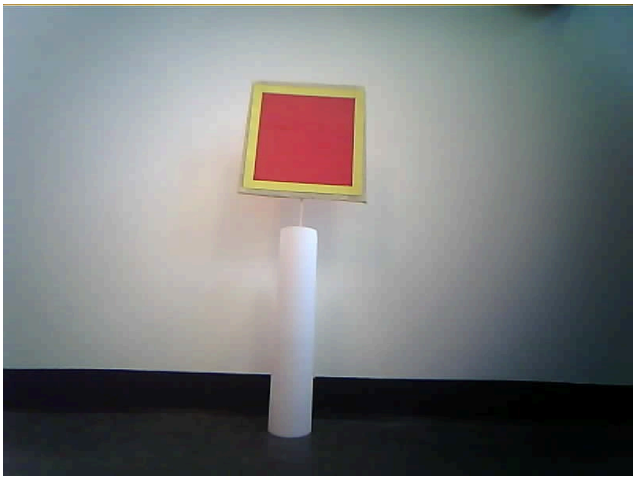
The external pipeline processes the camera stream to identify the task marker and determine its position within the image. The horizontal displacement of the detected marker from the centre of the camera frame is used to make fine adjustments that align the robot with the target. This process is part of the finite-state-machine controller in Section 3.5. Because the pipeline makes all navigation decisions, it also incorporates telemetry from the onboard IR and ultrasonic sensors into the same decision-making layer. The IR sensors provide boundary-marking information, while the ultrasonic sensor gives local obstacle information. This centralisation prevents independent control processes from generating potentially conflicting navigation commands.

The achievable frame rate depends on image resolution and quality, network bandwidth, and the ESP32's processing capabilities. Lower resolutions allow higher frame rates and reduce required network bandwidth, whereas increasing image resolution substantially increases the data that must be transmitted and processed. Increasing spatial image quality reduces temporal resolution. We characterised this trade-off experimentally by recording the camera stream at several resolutions. As shown in Fig. 3.7.1, the approximate frame rates were 26 FPS at QVGA (320×240), 13 FPS at SVGA (800×600), and 6 FPS at UXGA (1600×1200). For the experiments, we used SVGA because it provided sufficient spatial resolution at a frame rate suitable for navigation.

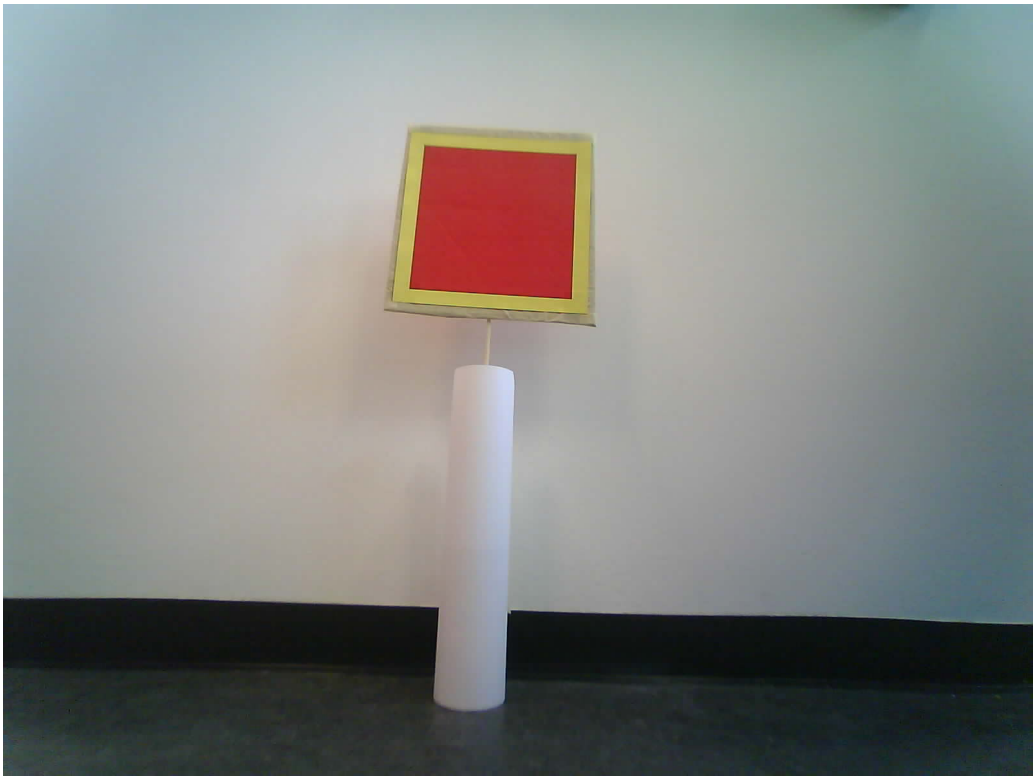
3.4 Communication Protocols and Information Flow

The experiment requires communication between the individual robots and the Python-based pipeline running on an external computer. This communication is divided into wireless communication, which connects each robot to the external computer and serial communication, which connects the wireless interface to the low-level Arduino controller.

(a) QVGA (320 × 240).



(b) SVGA (800 × 600).



(c) UXGA (1600 × 1200).

Figure 3.3.2: Camera images acquired at different resolutions and their corresponding approximate frame rates.

The information flow is

$$\text{Arduino} \overset{\text{UART}}{\longleftrightarrow} \text{ESP32} \overset{\text{WiFi}}{\longleftrightarrow} \text{Central Computer.} \tag{3.4.1}$$

3.4.1 WiFI and UDP Handshake

Each robot is assigned an IP address so the pipeline's computer can address it using an individual HTTP session and video-stream connection for each robot. Movement instructions are transmitted using HTTP requests to the robot's command endpoint. In the experimental implementation, the command is encoded as a request parameter to the robot's */blink* endpoint. The request is issued asynchronously in a separate thread, allowing the main control loop to continue processing camera frames and telemetry without waiting for the HTTP transaction to complete.

Sensor telemetry is transmitted via User Datagram Protocol (UDP) instead. A UDP socket provides a lightweight channel for transmitting telemetry data from the robots to the central computer with low communication overhead. A single UDP socket on the computer listens for incoming datagrams from all robots. The sender's IP address associates each telemetry packet with the corresponding robot.

When the system detects a previously unknown IP address, it creates a corresponding robot object. The robot then performs a telemetry handshake before the system considers it connected. A UDP handshake verifies that each robot is actively communicating with the computer before the experiment begins. The computer sends an initial command to the robot and waits for a UDP telemetry packet containing the robot's sensor data; once it receives valid telemetry, it marks the robot as connected. Subsequent commands are sent only after a functional communication link has been established.

Using UDP for telemetry avoids the overhead of maintaining HTTP connections and permits asynchronous transmission of frequent sensor measurements. This is suitable for real-time robot control.

3.4.2 Serial Communication via UART

Wi-Fi is used for communication with the external computer, while serial communication is used between the embedded microcontrollers. Within the robotic platform, the ESP32 communicates with the Arduino Uno using Universal Asynchronous Receiver–Transmitter (UART). UART is an asynchronous communication protocol in which data are transmitted sequentially over two dedicated communication lines: one for receiving and one for transmission [29].

UART does not require a shared clock signal between communicating devices [29]. Instead, both devices agree on a predefined transmission speed, called the baud rate. Data are exchanged as individual frames consisting of

- one start bit,

- a configurable number of data bits,
- an optional parity bit,
- one or more stop bits.

The asynchronous nature of UART considerably simplifies hardware implementation while giving reliable communication over short distances with minimal computational load [30].

3.5 Introduction to State Machine

A state machine, or more specifically a finite-state machine (FSM), is a mathematical model that describes systems whose behaviour can be represented by a finite number of discrete states and transitions between those states [31]. At any instant, the system occupies exactly one state, and transitions to another state only when a predefined event or condition is satisfied. An FSM is characterised by a set of states, an initial state, a set of inputs or events, and a transition function that determines how the system evolves in response to those inputs. Each state can correspond to different behaviour and represent the sequential execution of a task with fallbacks broken down into smaller steps.

3.6 Reverse Central Foraging Problem

Foraging is a standard test problem in multi-agent robotics, in which a group of robots repeatedly searches for resources and transports them to designated collection locations [32, 33]. In particular, the central foraging problem involves collecting resources from the environment and bringing them back to the central deposition area. This is common in primitive social insects like ants, which explore the environment and then bring resources back to their nest. These experiments involve individual decision-making, task allocation, environmental constraints, collision avoidance, and coordination among multiple agents.

In this work, we study a modified version of the conventional foraging problem, *reverse central foraging*. Rather than starting at a central deposition region and then searching for spatially distributed resources to bring back, the agents begin in a common central region, acquire task-specific information, and then travel to one of several spatially separated deposition regions. After completing the selected task, the agent returns to the central region. The central region therefore acts as a common point of departure and information acquisition, while the spatially separated regions represent distinct deposition regions. This arrangement implements a multi-task environment while retaining the repeated search–action–return structure of multi-agent foraging.

3.7 Experimental Layout

We implement the reverse foraging problem as follows. The problem involves collecting resources from the central regions, finding the appropriate deposition area, and then returning to the centre. Each robot must collect resources from the centre, learn the classification of each task, navigate to the marker, deposit, and return to the centre while avoiding obstacles and staying within the bounds of the experimental arena. To classify the given resource correctly, each robot runs a neural network as part of the pipeline. The implementation is described below.

3.7.1 AprilTag as Task Markers

As discussed in Sect. 3.3.3, a camera is a suitable primary sensory modality for both navigation and visual target detection. The experimental setup requires a set of unique visual markers, with each marker corresponding to a specific deposition region. These markers must remain reliably detectable at the onboard camera's spatial resolution and frame rate while maintaining sufficient localisation accuracy for the robot to navigate to the corresponding target.

We initially considered colour-based target detection. However, colour detection depends strongly on illumination conditions and the surrounding environment. In particular, an object's HSV representation can vary with lighting changes, potentially causing false detections or missed detections. Also, the experimental arena is surrounded by white walls. At larger distances, the white regions of small targets can become difficult to distinguish from the background due to limited image resolution and the resulting loss of spatial detail. This is particularly problematic for markers containing large white regions, such as conventional QR codes.

We used modified AprilTags [34] from the $36h11$ family as task markers. We modified the conventional black-and-white colour scheme by replacing the outer black region with green and the white interior with black. The outer green region has strong contrast against the white walls and the marker's black interior. AprilTags are designed to reliably identify and localise individual markers from camera images and are suitable for operation under relatively low-resolution imaging conditions [34]. The binary pattern identifies the marker directly, allowing each detected tag to be mapped to a deposition label.

Each AprilTag is assigned to a unique deposition label. Thus, the predicted label generated by the robot's neural network is associated with physical markers in the experimental arena. After receiving the resource or data and producing a label prediction, a robot uses its camera to search for the AprilTag corresponding to the predicted task. Once the marker is detected, the visual detection pipeline estimates its position in the camera image, which is then used to guide navigation toward the target while obstacle avoidance runs concurrently. The task is considered complete when the robot enters the predefined deposition region around the corresponding AprilTag.

3.7.2 Central Blue Cylinder

The task-specific AprilTags described in the previous section represent the target regions to which resources would be delivered in a reverse central-foraging scenario. Also, the experimental arena contains a central hub that serves as the common return location and resource collection region for the robots. The hub is positioned so it remains visible from much of the arena, allowing a robot to return to a known location after completing a task or when it cannot reach its assigned target within the allotted search time. The central region is therefore the reset point for the next task.

As discussed in Sect. 3.3.3 and the previous section, colour-based detection can be sensitive to changes in illumination and background conditions. Nevertheless, detection of a single, sufficiently distinctive colour can be made reliable by using a broad HSV threshold and by incorporating additional geometric constraints. We therefore selected a blue cylindrical object to represent the central hub along with a white horizontal strip placed around the centre of the cylinder to provide a distinctive geometric feature and reduce the likelihood of false detections arising from blue regions in the surrounding environment. The cylinder is also larger than the task markers, making it easier to detect when the robot is farther from the hub.

The return to the central hub forms the final stage of the task execution cycle. After the robot reaches its assigned AprilTag, or when the predefined search or navigation time expires, it switches from task-specific navigation to hub-search mode. The camera then locates the blue cylinder using the corresponding colour and geometric detection criteria. Once the robot detects the hub, it navigates toward it while continuing to avoid obstacles. Arrival at the central hub marks the completion of the current foraging cycle and returns the robot to an available state, allowing it to subsequently execute another task.

3.7.3 White Hexagon and Tape Markings

The experimental arena is defined using commercially available as masking tape. The tape serves two purposes: first, to define the physical boundaries of the arena and its individual regions; and second, to distinguish these regions using the infrared (IR) sensors mounted on the underside of the robots. The outer boundary of the experimental arena is formed using two layers of white masking tape. This surface produces an IR sensor response of approximately 150. Orange masking tape delineates the regions surrounding the task markers, yielding a measured IR response of approximately 250–370. The central hub is distinguished by a glossy white sheet of paper, which produces a lower IR response, typically below 60. These characteristic sensor responses define thresholds in the robot's control algorithm. These thresholds identify the robot's location and state transitions, such as reaching a deposition marker or returning to the central hub.

Masking tape was selected for the arena markings because its matte surface clearly distinguishes the different tape colours under the IR sensors and allows the arena to be constructed and modified easily. The outer boundary was arranged as a regular hexagon with a side length of approximately 180 cm. The hexagonal geometry was selected to

accommodate the six-robot, six-task configuration used in the experiments. Each task region is positioned at a vertex of the hexagon, creating a symmetric arrangement in which all task regions are approximately equidistant from the central hub. This symmetry minimises geometric differences between tasks and prevents a particular task from having an inherent positional advantage due to its distance from the centre of the arena.

Figure 3.7.1: Experimental Arena with resource markers at the vertex of the hexagons separated using coloured sheets and orange tape. The central blue cylinder is also visible on the white glossy circle.

3.8 Computational Pipeline

The computational pipeline consists of the Arduino and ESP32 code on the robots and the Python pipeline running on the external computer. The Python pipeline contains the AprilTag detector, navigation, and other components used during a foraging event.

3.8.1 Arduino and ESP32 code

As described above, the robot performs no local decision-making; sensory data is sent to the Python pipeline, and the response to the stimulus is sent back to the robot. Both microcontrollers are specifically programmed to make this possible. The ELEGOO manual includes code for all smart-car functions, including sensor pin connections [35]. On the Arduino, when the robot is switched on, the hardware initialises and all sensors enter operating mode. The MPU6050 is also calibrated during initialisation and should therefore be kept stationary for proper calibration. The predefined motion commands in [35] use the MPU6050 to compensate for deviation from initial headings. We kept the motor motion, sensory data collection, and sensor calibration at their default settings, as cited in [35].

The Arduino sends sensor measurements to the ESP32 at a fixed 50 ms interval. For each transmission, the sensory values are packed into an ArduinoJson document. The resulting message contains a *TELEMETRY* type field, the ultrasonic distance, the yaw angle rounded to two decimal places, and the three tracking-sensor values. A newline is appended after each JSON object and serves as the end-of-message delimiter for the ESP32. Commands are transmitted in the opposite direction using the same serial connection. The ESP32 sends commands, which the Arduino reads without blocking the main control function. After receiving a complete line, the instruction number is extracted and mapped to a motion. In the current implementation, the available commands correspond to forward motion, left and right turns, stopping, a right-hand scanning rotation, and backward motion.

The ESP32 has a UART interface configured for communication with the Arduino at 9600 baud, while its own serial interface is used for debugging at 115200 baud. The ESP32 also creates a mutex for the telemetry buffer, since the UART reader runs as a separate FreeRTOS task. The external computer running the pipeline sends motion commands through an HTTP server running on port 8080. The ESP32 returns an HTTP response immediately and then forwards the command to the Arduino through Serial2. The ESP32 also records the IP address of the client making the request, which it then uses as the destination for telemetry packets. This is done during the UDP handshake.

Telemetry from the Arduino is handled by a separate FreeRTOS task. The task continuously reads characters from the UART connection and accumulates them until it receives a newline or carriage return. The task then stores the completed message in the shared telemetry buffer and, once it knows the external computer's IP address, transmits it via UDP on port 4210. A mutex protects the shared telemetry string while it is being updated. The receive buffer is also limited to 512 characters; if it exceeds this limit, the incomplete message is discarded.

The ESP32 also handles the robot camera. The camera is configured to produce JPEG images at SVGA resolution (800×600). When PSRAM is available, the system uses two frame buffers and configures the camera to grab the latest available frame. During startup, the robot and the computer establish the wireless connection with Wi-Fi sleep

disabled. Once the connection is established, the camera server, HTTP command server, and UDP telemetry interface start. The UART reader is then created as a FreeRTOS task pinned to core 1 in the ESP32.

Arduino handles direct interaction with the robot hardware, while the ESP32 handles communication, camera access, and the interface to the external controller. The pipeline does not communicate directly with the motor drivers or sensors. It sends high-level movement commands to the ESP32 and receives the sensor state through the same ESP32 interface. The subsequent detection and navigation routines described in the following sections operate on top of this communication layer.

3.8.2 State Machine Architecture for Detection and Navigation

The robots' physical navigation is implemented as a state machine that combines visual target detection, infrared-based line detection, ultrasonic obstacle detection, and discrete motion commands. Each robot executes the navigation routine independently, using the most recent camera frame and sensor telemetry. The controller divides the navigation procedure into three main states. Each state is further divided into substates that distinguish between executing a movement and evaluating its result. The robot therefore alternates between short movements and sensor observations.

Before each navigation step, the controller checks the infrared sensors and ultrasonic distance measurements. These checks have priority over the normal navigation state machine. A detected boundary condition can therefore interrupt the current navigation action, while an obstacle causes the robot to enter a separate avoidance procedure. The camera frame is then processed to determine whether the requested target is visible and, if so, its position relative to the centre of the image is used to determine the next motion command. The task terminates when the target is reached or when the predefined task timeout is exceeded.

Object Avoidance

Obstacle avoidance is based on the robot's ultrasonic distance measurement. An obstacle is considered to be present when the measured distance falls below 20 cm. To avoid reacting to a single erroneous measurement due to acoustic crosstalk, the distance has to remain below this threshold for at least 0.3 s before an obstacle is confirmed. If the distance subsequently rises above the clearance threshold, the obstacle state is cleared.

Once an obstacle has been confirmed, the normal target-navigation state is temporarily suspended. The robot first moves backwards until either the measured distance becomes sufficiently large or a maximum backwards-motion time of 1.2 s is reached. It then stops briefly and performs a rotational scan. Following the scan, the controller checks the distance again. If the path is still obstructed, it performs another scan; otherwise, the robot moves forward for a fixed period of 1 s. During this forward motion, the distance sensor continues to be checked, so that a newly detected obstacle causes the avoidance sequence

to restart.

The obstacle routine is implemented as a sequence of internal steps rather than as a single blocking operation. This allows the distance measurement to be evaluated between actions and prevents the robot from committing to a complete avoidance manoeuvre without rechecking the environment. After the forward movement has completed without detecting another obstacle, the avoidance state is cleared and normal target navigation resumes.

Line Detection

The robot uses three infrared sensors, positioned on the left, centre, and right, to detect the coloured boundary lines in the arena. The robot receives the sensor values as part of the telemetry and classifies them into three ranges corresponding to white, glossy white, and orange surfaces. A colour is considered detected when at least two of the three sensors simultaneously fall within its corresponding range.

White lines are treated as arena boundaries. When the robot detects a white line, it immediately stops, moves backwards for a fixed duration, and then stops again. The navigation state is subsequently reset to the scanning state. This prevents the robot from continuing towards and potentially crossing the arena boundary.

Glossy white and orange lines are used as task-dependent completion signals. For a cylinder-return task, detection of a glossy white line causes the robot to stop and the task to be considered complete. For a marker task, an orange line is recorded as a possible completion signal, but the line detection alone does not terminate the task. Instead, the orange signal is combined with the visual marker detection, preventing an isolated infrared measurement from being interpreted as successful completion.

AprilTag Detector

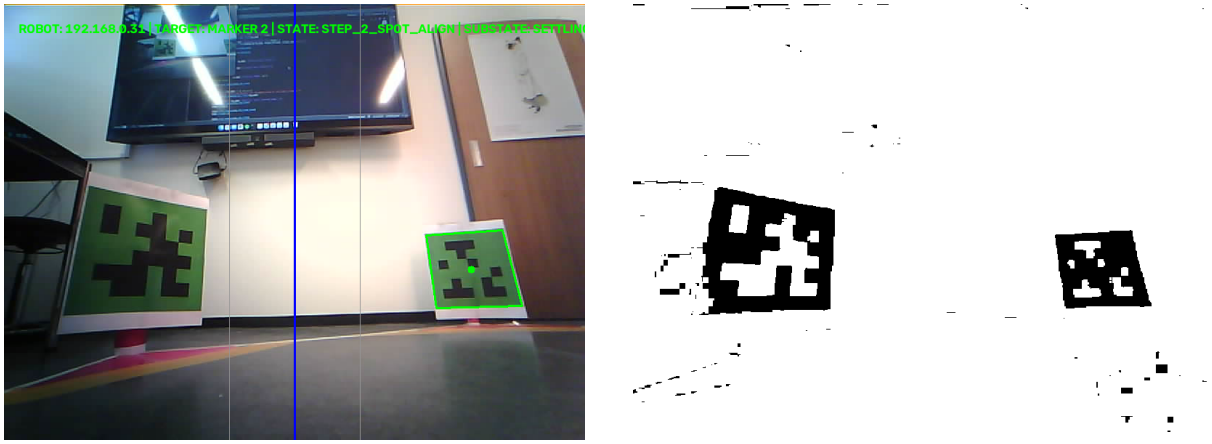
Each robot maintains a separate detector instance, with one detection thread assigned to each detector. The detector operates on the full-resolution image, without image decimation, and uses edge refinement and image-sharpening parameters to improve detection.

Before AprilTag detection, the camera image is converted from BGR to HSV colour space and thresholded using a green colour range. Morphological closing is applied to the resulting mask, filling any gaps in the pixel data, before calling the AprilTag detector. The processed mask is inverted and supplied to the detector. The preprocessing uses the coloured outer region of the modified marker while retaining the black-and-white structure required by the AprilTag detector.

During normal target search, only detections with the requested tag ID are accepted. Additional geometric checks reject detections with very small image area or bounding boxes. For an accepted detection, the detector returns the tag centre, bounding box, corner coordinates, horizontal image error, detection area, decision margin, and Hamming

distance. The horizontal error is calculated relative to the centre of the camera image and is subsequently used by the navigation state machine for alignment.
If the robot gets too close to the marker or decoding becomes difficult, the detector uses a local contour-based fallback to prevent losing the target. If the AprilTag detector temporarily fails after a target has already been detected, the previous bounding box defines a local region of interest. The system then examines green contours within this region, allowing the robot to maintain a lock on the target based on its coloured outline.

The same detection interface is used for both task markers and the central hub. Marker targets are identified through their AprilTag ID, while the blue-and-white cylinder is detected separately using colour segmentation and geometric constraints on the detected blue regions.



(a) Raw frame from camera (b) Processed frame for AprilTag Detector

Figure 3.8.1: (a) Contains the raw frame captured by the OV2640 camera. (b) The post-processed frame used by the AprilTag detector.

Navigation

The navigation procedure is organised into three principal states. In *STEP 1: SCAN*, the robot searches for the target by rotating in discrete increments rather than continuously. Each scan consists of a rotation followed by a settling period, after which the robot examines a new camera frame. Six such scan steps correspond to one complete revolution. If the robot detects the target during a scan, it stops rotating and transitions to the alignment state.

In *STEP 2: SPOT ALIGN*, the robot uses the detected target's horizontal position to align with it. The controller defines a central tolerance region around the image centre. If the target lies outside this region, the robot performs a short rotational nudge towards the target. The nudge duration decreases as the horizontal error decreases, giving a smaller correction for small misalignments. After each correction, the robot stops and obtains another observation. Once the target is sufficiently close to the image centre, the robot transitions to the forward-motion state.

The third state, *STEP 3: STEP AND CHECK*, implements the forward approach. The robot moves forward for a short, predefined interval and then stops to obtain a new observation. If the target remains visible but is displaced from the centre, the robot applies a rotational correction before the next forward step. If the target is no longer visible, the robot stops and returns to the scanning state *STEP 1: SCAN* after a specified period without detection. This step-and-check procedure uses a new visual observation after each forward movement.

Target completion is determined from the apparent size of the detected marker. For the requested marker, the task is completed when its detected bounding-box width exceeds the predefined threshold of 500 px. Only the originally requested tag ID can trigger completion, including during the fallback search described below. A separate timeout terminates the task if the marker has not been reached within the allowed duration.

If the controller cannot detect the requested marker for 25 s, it temporarily enters fallback search mode. During the fallback, the robot searches the requested marker and its two neighbouring markers simultaneously. If it detects a neighbouring marker, the robot can briefly approach it before returning to the search procedure for the original marker. Subsequent fallback cycles search only for the originally requested marker. This is useful when the marker is far away or is obstructed by other agents or the central cylinder.

The navigation routine consists of repeated cycles of *observe, decide, move, and observe again*. The robot does not rely on a predetermined path through the arena. Instead, the direction and duration of each movement are determined from the most recent target position and sensor measurements. Each robot's complete navigation routine is protected by a hardware lock and includes an emergency stop in the termination path, so that the motors are commanded to stop when the task finishes or an error occurs.

3.8.3 Robot Tracking and Trajectory Calculation

We reconstruct the robot trajectories from the experimental video using a perspective-corrected Kanade–Lucas–Tomasi (KLT) feature-tracking procedure [36–38]. The four corners of the video frame are selected in the original video to calculate a planar homography to transform the recorded frames into a rectified coordinate system. Each robot is attached with unique AprilTags in the default black-and-white colours, making them distinguishable "good features for detection and tracking.

The six robots are manually identified in the first rectified frame, thereby establishing their persistent identities $R_1, \ldots, R_6$. Around each selected robot, we define a local region of interest (ROI) of approximately 60×60 pixels, within which we detect distinctive image features using the Shi–Tomasi "good features to track" criterion [37]. These features provide the points used for subsequent frame-to-frame tracking.

We then estimate each robot's motion using the pyramidal KLT optical-flow method im-

plemented in OpenCV's *calcOpticalFlowPyrLK*. The method estimates the displacement of local image features between consecutive grayscale frames by minimising their local intensity difference, while the image-pyramid formulation allows larger displacements to be recovered more reliablely [36, 37]. To remove incorrectly tracked features, we apply a forward–backward consistency check: features are tracked from the previous frame to the current frame and then back to the previous frame, and we discard features whose round-trip error exceeds 1.5 pixels. We then use the median displacement of the remaining features to estimate the robot's translational trajectory in the experimental arena.

The tracking procedure also incorporates adaptive feature refreshing and manual reinitialisation. When the number of reliably tracked features falls below 12, the system detects new features within an enlarged ROI around the current robot position. Tracking can continue when individual features are lost because of changes in illumination, motion, or appearance. Also, the robot identities can be manually re-selected periodically (by default every 20,000 frames) or whenever required, with the newly selected positions used to reinitialise the KLT features while preserving the corresponding robot identities and accumulated trajectories. For each frame, we record the estimated robot centre (x, y), frame number, time, tracking status, and number of successfully tracked features. The sequence of centre positions gives the two-dimensional trajectory of each robot throughout the experiment in the perspective-corrected arena coordinate system.

Chapter 4

Results

4.1 Numerical Realisation of the Decentralised Learning Ensemble

This section discusses the results of the numerical framework developed by Pfaffenzeller [6]. The original work explored whether self-organisation at the ensemble level can address the stability-plasticity dilemma of continual learning. The section introduces the specific experimental design adopted and its results. The results show that stable and adaptive task specialisation can emerge from local, performance-based interaction without a central coordinator [6].

4.1.1 The Collective Solution to the Stability-Plasticity Dilemma

As discussed earlier, in continual learning, cross-task interference can cause phenomena such as catastrophic forgetting, and conventional multi-agent systems require a centralised controller or coordinator to allocate tasks across an ensemble of agents. Pfaffenzeller proposed a self-organised mechanism that allows an ensemble of agents to compete to specialise in different tasks. In this system, memory arises at the system level while plasticity is retained because task assignment is not fixed and the system can respecialise.

The system was inspired by decentralised biological systems like the *Polistes* wasp colonies, where stable queen-worker differentiation and emergent reorganisation in response to queen removal can emerge from local interactions [39–41].

4.1.2 System Architecture and State Representation

The ensemble consists of N neural agents that solve M classification tasks. Each agent i is characterised by $(w_i, \mathbf{s}_i, \mathbf{a}_i)$, where w_i denotes the neural-network parameters, $\mathbf{s}_i = (s_i^1, \ldots, s_i^M)$ are binary task-activity variables, and $\mathbf{a}_i = (a_i^1, \ldots, a_i^M)$ are task-specific accuracies [6]. The activity variables can take the value:

$$s_i^m \in \{0, 1\}, \tag{4.1.1}$$

where $s_i^m = 1$ indicates that training on task m is currently allowed and $s_i^m = 0$ indicates temporary suppression. These suppressive interactions do not directly alter the network's parameters. They regulate access to training opportunities. Hence, suppression indirectly reshapes performance by controlling the sequence of training opportunities the agent experiences.

Each of the N agents is a fully connected feed-forward neural network with the architecture

$$d \to 128 \to 64 \to K, \tag{4.1.2}$$

where d is the input dimension and K is the number of classes. The system weights were initialised using Glorot (Xavier) uniform initialisation [42]. Training used the Adam optimiser and categorical cross-entropy loss, using a small fixed learning rate to ensure gradual performance evolution [6].

4.1.3 Synthetic Gaussian Training Data and Task Orthogonality

To enforce cross-task interference and study the system under controlled conditions, we trained the ensemble on synthetic multi-class classification tasks defined in a d-dimensional feature space. For each simulation run, a reference dataset was generated by first sampling K class centres uniformly

$$\boldsymbol{\mu}_k \sim \mathcal{U}([-1, 1]^d), \qquad k = 1, \ldots, K. \tag{4.1.3}$$

We assigned an isotropic Gaussian distribution to each class. The variance of this distribution was sampled as

$$\sigma_k^2 \sim \mathcal{U}[0.01, 0.05], \tag{4.1.4}$$

from which individual samples were drawn according to

$$\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}_k, \sigma_k^2 \mathbf{I}_d). \tag{4.1.5}$$

Deterministic cyclic permutations were applied to the class labels of this synthetic data while keeping the input samples unchanged. This created a conflicting input-label mapping, preventing knowledge learned on one task from transferring to another and causing interference between tasks.

4.1.4 Stochastic Event-Driven Gating Dynamics

The continuous-time dynamics were implemented using a Gillespie Simulation Algorithm where each agent could undergo two classes of events:

1. **Training events** (i, m), which update the parameters of agent i using task m.
2. **Suppression events** $(i \leftarrow j, m)$, in which agent j temporarily disables training of agent i on task m.

While being suppressed on one task, the agent could still train on others. The training events occurred at a fixed training rate

$$\lambda_i^m = \lambda_{\text{tr}} s_i^m, \tag{4.1.6}$$

The rate of suppression events is performance-dependent and given by

$$\lambda_{i\leftarrow j}^m = \alpha a_i^m a_j^m \sigma\left[\eta(a_j^m - a_i^m)\right], \tag{4.1.7}$$

with the sigmoid function

$$\sigma(x) = \frac{1}{1+e^{-x}}. \tag{4.1.8}$$

The prefactor $a_i^m a_j^m$ ensures the strongest suppression occurs when both agents compete on the same task by maintaining high accuracy, while the sigmoid term introduces asymmetry that biases the higher-performing agent. The parameter η controls the sharpness of this asymmetry, and α sets the overall suppression frequency. A suppression event sets $s_i^m = 0$ for a refractory time τ, after which the task becomes active again. In the Gillespie simulation, training and suppression events are sampled according to their instantaneous propensities. The simulation loop is summarised below.

4.1.5 Simulation Results

Figure 4.1.1 shows the results for $N = M = 10$ using strong suppression and highly asymmetric interactions $\alpha = \tau = 10^3$, $\eta = 10^6$), a regime in which stable specialisation is observed [6]. The plot shows the time evolution of the agent performance in a barycentric radar representation, with each vertex corresponding to a task. Initially, all agents are near the centre, representing low accuracy on all tasks; over time, their trajectories move outward toward different vertices, showing the emergence of specialisation.

Competition between agents appears at intermediate times, when multiple agents approach common task vertices, but suppression events gate training opportunities, eliminating the competitor and redirecting it to other available tasks. At the final times of the simulation, the trajectories remain localised near the vertices, indicating dynamical stability
After this sorting process, the trajectories remain localised near distinct vertices with only small fluctuations, indicating that the specialised configuration is dynamically stable over long times. These simulations show that the feedback loop between training-induced performance changes and performance-dependent suppression is sufficient to generate division of labour within the ensemble, without any externally imposed task assignment [6].

4.1.6 Characterization of Phase Space and Dynamical Regimes

The specialisation depends on the chosen hyperparameter values, and their combinations introduce different learning regimes. To investigate this, Pfaffenzeller performed systematic parameter sweeps over the product of the suppression-rate prefactor and the refractory

Algorithm 1: Gillespie simulation loop

Initialize networks w_i, activities $s_i^m \leftarrow 1$, and time $t \leftarrow 0$;
Compute initial accuracy matrix $A(0)$;
while $t < T$ **do**
  forall *suppressed pairs* (i, m) **do**
    if $t - t_0^{(i,m)} \geq \tau$ **then**
      $s_i^m \leftarrow 1$;
    end
  end
  Compute training propensities $\lambda_i^m = \lambda_{\text{tr}} s_i^m$;
  Compute suppression propensities

$$\lambda_{i \leftarrow j}^m = \alpha a_i^m a_j^m \sigma\big[\eta(a_j^m - a_i^m)\big]$$

  Form total propensity Λ;
  Draw waiting time

$$\Delta t = -\frac{\ln u_1}{\Lambda}$$

  Select one event with probability proportional to its propensity;
  $t \leftarrow t + \Delta t$;
  if *event is training* (i, m) **then**
    Train agent i on task m;
    Update w_i and recompute accuracies;
  end
  else
    $s_i^m \leftarrow 0$;
    $t_0^{(i,m)} \leftarrow t$;
  end
  Record observables $A(t)$ and $S(t)$;
end

time $\alpha\tau$ and the interaction asymmetry [6]. The resulting phase diagram, shown in Fig. 4.1.2, is quantified using the normalized task-coverage metric

$$\frac{C_{\geq\theta}(T)}{M}, \tag{4.1.9}$$

where $C_{\geq\theta}(T)$ counts the number of tasks with at least one unique agent achieving accuracy above the threshold $\theta = 0.9$ at the final simulation time T [6].
Fig. 4.1.2 establishes the three qualitative learning[6]:

1. **Training-dominated unspecialised regime** ($\alpha\tau \ll 1$): suppression is too weak to prevent agents from training multiple tasks, and persistent specialisation does not emerge. Figure 4.1.3a shows the radar plot for this regime.

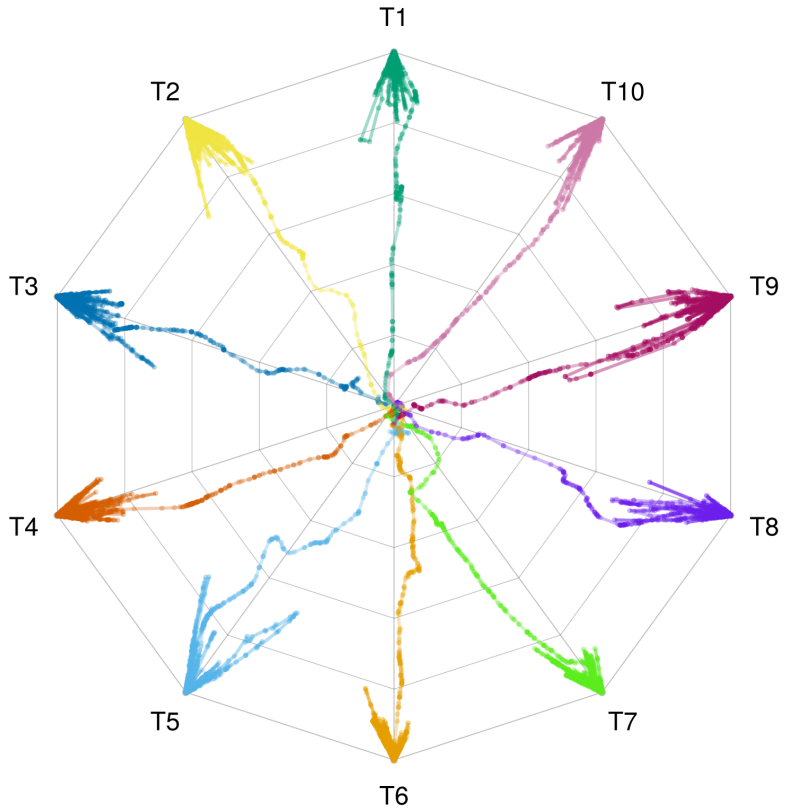


Figure 4.1.1: Representative realization of the decentralized learning ensemble for $N = M = 10$ in a strong-suppression regime ($\alpha = \tau = 10^3$, $\eta = 10^6$). Each trajectory corresponds to one agent in the barycentric radar representation; movement toward a vertex indicates increasing specialisation on the corresponding task.

2. **Specializing regime** (large $\alpha\tau$ with large η): agents self-organize into stable one-agent–one-task configurations with near-complete task coverage.

3. **Suppression-dominated regime** (very large $\alpha\tau$ with small η): training opportunities become excessively rare, leading to loss of task coverage. Figure 4.1.3b shows how strong symmetric suppression leads to repeated inactivity on tasks and a mutual lock.

4.1.7 Ensemble Plasticity and Reconfigurability

The plots above show the emergence of memory at the ensemble level. Pfaffenzeller 2026 also showed that the system exhibits adaptive reorganisation at the ensemble level [6]. Two perturbation protocols were studied.

Task addition. To test whether the neural ensemble can incorporate new tasks without external reassignment, additional tasks were introduced after the system had specialised. Figure 4.1.4 shows the perturbation experiment of Ref. [6]. The simulation was initialised with $N = 15$ agents and $M = 10$ tasks, with ten agents specialising in 10 distinct tasks

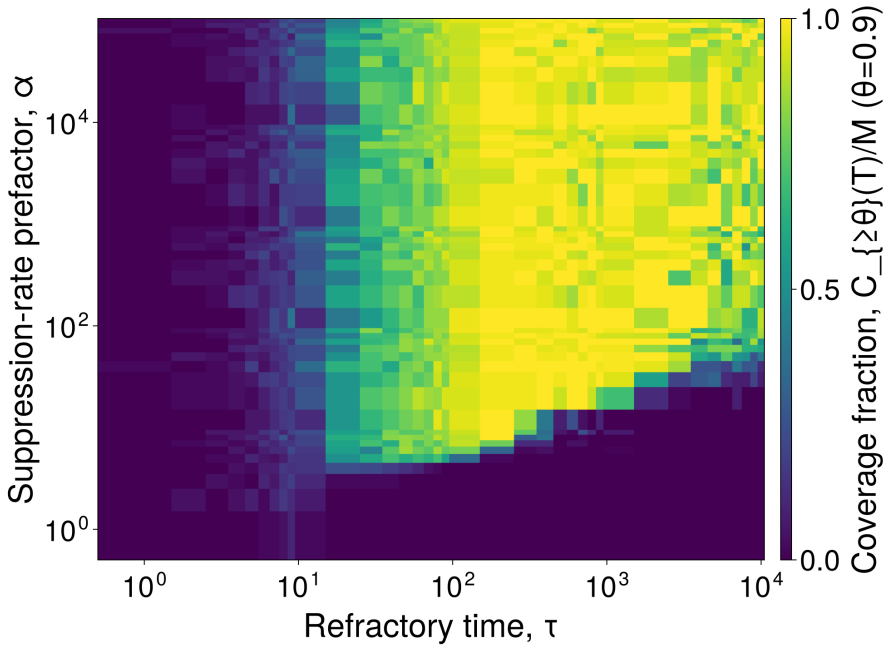
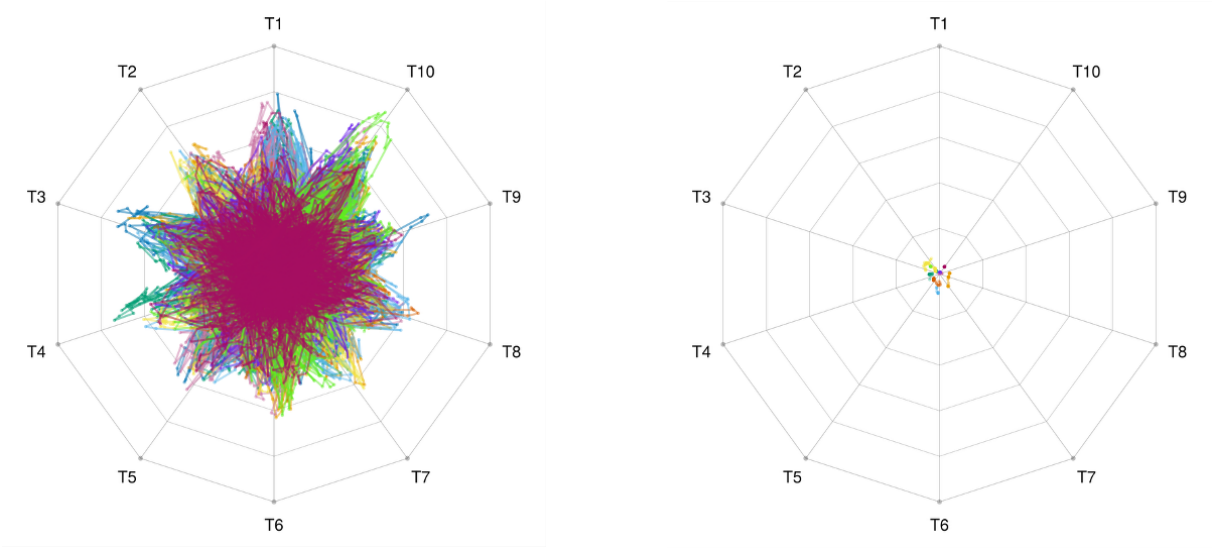


Figure 4.1.2: Phase diagram of the decentralised learning ensemble as a function of suppression frequency α and refractory time τ at fixed interaction asymmetry $\eta = 10^6$. Colors indicate the normalized task coverage $C_{\geq\theta}(T)/M$ with threshold $\theta = 0.9$. The dimensionless suppression-strength product $\alpha\tau$ approximately organises the transition. Adapted from Ref. [6].

while the rest of the agents remained unspecialised (Fig. 4.1.4a). After five new tasks are added, the previously non-specialised agents begin to move away from the origin and explore the newly available task directions (Fig. 4.1.4b). Continued evolution leads to a new specialised configuration with all tasks being occupied by unique agents (Fig. 4.1.4c). This experiment demonstrates that the spare agent capacity can be recruited dynamically, and the same decentralised suppression-and-learning mechanism that generated the original specialisation is sufficient to reorganise the system and restore complete task coverage after expanding the task set [6].

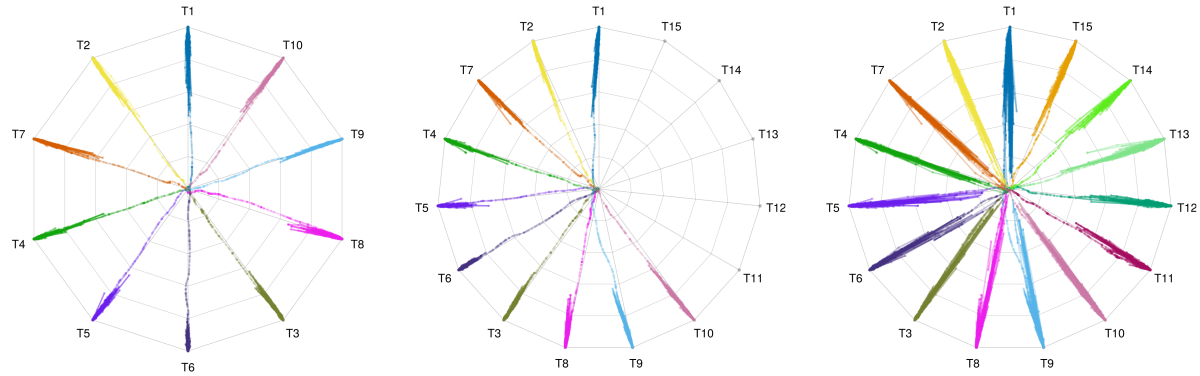
Agent removal. A complementary perturbation experiment involves removing specialised agents from the ensemble. Figure 4.1.5 reproduces the corresponding experiment from Ref. [6]. Starting from a specialised configuration with $N = 15$ agents and $M = 10$ tasks (Fig. 4.1.5a), five agents are removed randomly, leaving only ten agents in the system (Fig. 4.1.5b). Immediately after the perturbation, several task vertices become vacant, temporarily reducing task coverage. With time, as the system is allowed to evolve with the same parameters, the remaining agents redistribute across the available tasks and eventually recover a fully specialised configuration (Fig. 4.1.5c). This recovery occurs without any centralised reassignment procedure and arises purely from interactions and system dynamics.
deter
Together, these perturbation experiments show that the decentralised learning ensemble exhibits both *stability* and *plasticity*. Stable task specialisation is maintained under steady



(a) Example of training-dominated unspecialised state

(b) Example of suppression-dominated regime

Figure 4.1.3: Radar Plots of Different Learning Regimes Adapted from Ref. [6].

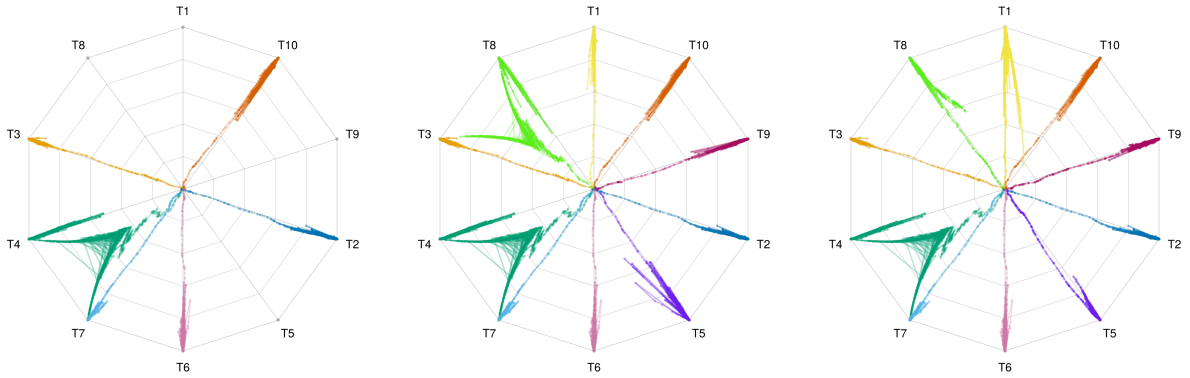


(a) Specialised state for $M = 10$.

(b) Immediately after adding five tasks.

(c) Reorganised specialised state for $M = 15$.

Figure 4.1.4: Adaptive reorganisation after introducing five additional tasks. Adapted from Ref. [6].



(a) Specialised state before perturbation.

(b) Remaining agents immediately after removal.

(c) Recovery of full specialisation after continued evolution.

Figure 4.1.5: Adaptive reorganisation after random removal of five agents. Adapted from Ref. [6].

conditions, while the ensemble can recruit unused agents, reallocate task responsibilities, and recover task coverage after substantial structural perturbations.

4.2 Self-organisised Specialisation in Robotic Ensembles

4.2.1 Environment-Dependent Event Gating in Physical Multi-Agent Systems

In Sect. 4.1, we introduced a system of interacting neural networks in which competitive interactions give rise to role specialisation, and demonstrated how this mechanism can provide a self-organised solution to the stability-plasticity dilemma. This establishes that stable memory and plasticity can emerge from local interactions between interacting agents. The framework can be extended to intelligent robotic collectives in standard experiments like foraging. Such an implementation aims not only to test the specialisation mechanism under physical constraints, but also to investigate whether emergent roles can be maintained amid perturbations and variability in the physical environment.

A minimal implementation outsources the robot's computational workload and decision-making to an external computer. Physical resources require physical collections, temporary storage on the robot, and physical depositions. Instead, the robot uses digital resources, which it can collect only once it satisfies criteria tied to the experiment's physical setup. This digital resource includes synthetic data and the label of the designated deposition area. The synthetic data serves as training data for the robot's neural network, and the label is used to evaluate the network.

Synthetic data has the advantage that no additional observables need to be developed as proxies for network performance. Instead, we can measure each neural network's per-

formance directly through its classification accuracy. Furthermore, separating task space from physical space provides greater flexibility in the experimental design, since different design configurations can be introduced without requiring the physical environment itself to uniquely encode the tasks. However, care must be taken to ensure that the resulting experiment does not become merely "symbolic". That is, the robots should not constitute a physical realisation of a numerical simulation in which computational events depend in no meaningful way on the physical environment. Such a setup would not rigorously test the framework introduced in Sect. 4.1, since the robots' actions and computational events would remain effectively independent of the physical space.

To avoid this, we explicitly trigger computational events through the robot's physical state or through some physical cue in the experimental arena. Hence, even if a robot is scheduled for foraging, its performance is not guaranteed to improve until it completes the task by meeting all the requirements.

When a robot is scheduled for a foraging event, it not only collects physical goods in the respective area but also trains and evaluates its neural network to learn what label each digital resource. The foraging event proceeds as follows: (1) Verifies the central blue cylinder is in its FOV; only then it receives the respective resource containing training data. (2) The robot's network trains once on the training data and predicts the label of the resource. (3) The robot starts searching for the AprilTag marker corresponding to that resource. (4) Uses the finite state machine logic to reach the target. (5) If the robot successfully reaches the marker, the network's performance on all tasks is evaluated. (6) In the case where the robot reached the marker successfully or the given time for foraging is over, the robot returns to the blue cylinder.

In this workflow, resource collection can happen only if the robot can see the blue cylinder, and model evaluation can happen only if the robot reaches the marker. If the robot cannot reach the marker within the given time, its performance does not improve and remains the same as before the foraging event, thereby influencing its future foraging opportunities. If the blue cylinder is not seen within a given time, the foraging event is cancelled after a fixed time. By conditioning training and evaluation on events within the physical experimental arena, the computational dynamics become directly coupled to the system's physical constraints.

The temporal scheduling of the robotic agents also needs to be considered. Under the standard Gillespie algorithm, events are scheduled sequentially, such that only one robot executes a foraging event at a given time. Since each foraging event requires a finite amount of physical time, a sequential implementation would result in prohibitively long experimental runs, with the duration increasing with the number of robots in the system. Moreover, because the robots remain powered throughout the experiment, battery capacity adds another practical constraint. Sequential execution is therefore unsuitable for experiments involving larger multi-agent systems.

The τ-leaping approach introduced in Sect. **??** is therefore used. Unlike the standard

Gillespie algorithm, τ-leaping allows multiple events to occur within a finite interval τ, enabling multiple robots to execute their activities concurrently. This parallel execution substantially shortens experimental runs and makes larger systems feasible to test. Furthermore, simultaneous operation introduces local interactions and stochastic variability arising from the physical environment, providing an additional source of fluctuations against which the robustness of the emergent specialisation mechanism can be tested.

4.2.2 Training data

The digital resource consists of a family of synthetic classification tasks constructed from a common underlying dataset. The data are generated in a d-dimensional feature space by associating each class with a multivariate Gaussian distribution. For a dataset containing K classes, the centre of each class, $\boldsymbol{\mu}_k \in \mathbb{R}^d$, is sampled independently from a uniform distribution within the prescribed bounds,

$$\boldsymbol{\mu}_k \sim \mathcal{U}([-1, 1]^d). \tag{4.2.1}$$

The resulting class centres define the underlying feature distribution from which the training and test samples are drawn.

Pfaffenzeller [6] used this underlying dataset to construct multiple tasks by deterministically rotating the class labels. Consequently, all tasks shared the same input features, but the correspondence between input samples and target labels differed across tasks. This construction introduced cross-task interference: learning to improve performance on one task could simultaneously reduce performance on another, since the same input samples were associated with different target outputs. We assigned labels so each task corresponded to a distinct label, with task labels ranging from 1 to the total number of tasks.

For the experimental implementation presented here, we needed to modify this task construction for resources. Each resource needed to correspond to a unique physical marker so the neural network output could be directly mapped to a specific target in the robotic experiment. Therefore, rather than applying a deterministic rotation to the labels, we set the labels of all samples within a given task to a single task-specific value. Thus, for task k, every input sample was assigned the label k,

$$y_i^{(k)} = k \qquad \forall i. \tag{4.2.2}$$

For example, all samples in task 1 receive label 1, while all samples in task 2 receive label 2. The input features therefore remain identical across tasks, while the desired output is uniquely determined by the task.

This construction preserves the cross-task interference characteristic of the approach introduced by Pfaffenzeller [6], since the same input features are associated with different target labels by task. At the same time, it provides a direct correspondence between a learned task and a physical marker: the neural network's output label uniquely identifies the marker associated with that task. This allows the synthetic learning environment to be transferred directly to the physical experiment while retaining the competing learning

objectives required to investigate task specialisation. Pfaffenzeller's tasks are modified as resources for the robots.

4.2.3 Foraging Event Sequence

The preceding sections described the individual components of the experimental setup and briefly outlined the foraging event. Here, all individual components come together to describe the complete foraging event and the sequence of sensory detections, navigation actions, training, and evaluation that constitute a single event.

When a robot is scheduled to perform a foraging event, it first verifies that the central hub is within its field of view. Once the hub is detected, the robot receives the resource data associated with the scheduled task and performs one training update. The trained network then produces a task prediction, which determines the target the robot will navigate toward. At the beginning of the experiment, the networks are untrained and therefore provide effectively uninformative task predictions. As learning progresses, the predictions become increasingly influenced by the accumulated training history and the system's interaction dynamics.

The predicted resource label is mapped to its corresponding AprilTag in the experimental arena. The robot then enters a target-search phase, during which it scans the arena for the corresponding marker. Once the robot detects the marker, it navigates toward it while continuously processing camera and IR sensor measurements. The robot determines arrival at the target using the predefined marker-region criteria. In particular, the robot is considered to have reached the target when it detects the orange boundary marking associated with the task region and/or when the detected target reaches the prescribed image size, corresponding to a pixel width exceeding 500 pixels. Reaching the target terminates the target-search phase and triggers an evaluation of the robot's performance across all available tasks. The learning and interaction dynamics then use the resulting performance measurements to determine the robot's state for future events.

A target may not always be successfully reached. For example, another robot or the central hub may temporarily occlude the corresponding AprilTag. Detection can also become difficult when the robot approaches the target from an unfavourable viewing angle or when the marker occupies only a small portion of the camera image. To account for such situations, the target-search phase is subject to a predefined temporal limit. If the robot cannot locate the corresponding marker within this period, it abandons the target search and initiates the return phase. In this case, the robot does not update its current performance and retains its previously measured values.

After successful target acquisition or when the target-search period expires, the robot enters the return phase. The robot searches for the central hub using the blue cylinder as the visual reference and, once detected, navigates towards it. The robot continues returning until it detects the characteristic IR response of the glossy white surface surrounding the central hub. Detection of this surface indicates that the robot has reached the cen-

tral hub and terminates the current foraging event. The robot is then returned to the available state and can subsequently be scheduled for another training and foraging event.

Unlike the target-search phase, the return phase has no temporal limit. The central hub is deliberately positioned and designed to remain detectable throughout the experimental arena, providing a common reference point to which robots can return regardless of the outcome of the preceding target-search phase. The absence of a temporal constraint on the return phase ensures that a robot is not prematurely released for another event while it is still navigating back to the common starting region.

The complete foraging event can therefore be summarised as a sequence of *collection*, *training*, *label prediction*, *target search*, *target navigation*, *performance evaluation*, and *return*. The following section describes the navigation algorithms and the specific sensory criteria used to control the robot during these phases.

4.2.4 τ-leaping based event-driven dynamics

The previous section introduced τ-leaping and its use for running robots in parallel. This section describes event scheduling and parallel task execution with robots.

The implementation maintains each agent's performance as the main state variable. In addition, an activity matrix records whether a particular agent–task pair is currently active, while the suppression time is stored for inactive pairs. Each agent has its own neural network, optimiser, and data iterators, allowing the learning process to proceed independently. At the start of each τ interval, the current performance matrix determines the propensities for possible events. Foraging events are generated from the fixed production rate, whereas suppression events depend on the interaction between two agents through the interaction kernel introduced in Eq. 4.1.7. Suppressed agent–task pairs are subsequently reactivated once their prescribed dead time has elapsed.

The sampled events are then converted into explicit event objects containing the event type, the agent involved, the task, and, for suppression events, the interacting agent. We sample the number of occurrences of each event for the current τ interval, and assign the resulting events to the corresponding agents. This separates the stochastic scheduling of events from their execution, which is particularly important for the physical experiments since an event may involve several seconds of task execution by a robot.

For a foraging event, the corresponding robot collects and deposits the resources. If the foraging was successful, the neural network is evaluated, and the resulting performance updates the corresponding entry in the performance matrix. Suppression events do not involve physical robot movement. Instead, the corresponding agent–task pair is marked inactive, and the time suppression began is recorded. This prevents further foraging events for that pair until the dead time has elapsed. Once the recovery condition is satisfied, the pair is returned to the active state and can again participate in the event dynamics.

A key aspect of the implementation is that events belonging to different agents can be executed concurrently. After sampling event counts for a τ interval, the events are grouped by agent and executed using a thread pool with one worker per agent. Consequently, two or more robots can carry out their respective physical actions at the same time instead of being forced to execute sequentially. Locks protect access to shared variables, particularly the performance matrix and activity state. After processing all events assigned to the agents for the current interval, the simulation time advances by τ, and the next set of event propensities is calculated.

The process can be summarised as follows:

1. **Check for recovery:** Check all currently suppressed agent-task pairs. If the corresponding suppression dead time has elapsed, reactivate the pair.

2. **Calculate event propensities:** Use the current performance matrix and activity state to calculate the propensities for training and suppression events.

3. **Sample events:** For each possible event, sample the number of occurrences within the current τ interval from the corresponding Poisson distribution.

4. **Assign events to agents:** Convert the sampled events into individual events containing the event type, agent, and task. Then assign the events to their corresponding agents.

5. **Execute events in parallel:** Events assigned to different agents are executed concurrently. Each agent processes its assigned sequence of events within the current τ interval.

   Detect the central hub: For a training event, the robot first searches for the central blue cylinder. Training can proceed only after the hub is detected, since it marks the location where the training data are available.

   Train the neural network: Once the hub has been detected, the agent performs the corresponding training step using the task-specific training data.

   Perform physical foraging: The robot leaves the central hub and performs the foraging procedure associated with the selected task. It searches for and navigates towards the corresponding task marker.

   Return to the hub: After completing the outbound foraging procedure, the robot returns to the central hub.

   Evaluate and update performance: Evaluate the trained model and update the corresponding entry in the performance matrix with the resulting performance. The updated performance is then available to the event dynamics for subsequent intervals.

6. **Update the overall performance matrix:** Once the events assigned to the agents are complete, collect the updated performance values from the individual agents and incorporate them into the shared performance matrix.

7. **Apply suppression events:** If a suppression event occurs, mark the corresponding agent–task pair as inactive and record its suppression start time. No physical robot action is required for a suppression event.

8. **Synchronise the interval:** After the events assigned to the agents for the current interval have been processed, the shared state is synchronised and the simulation time is advanced by τ.

9. **Start the next interval:** The updated performance and activity states are used to calculate the propensities for the next τ interval, and the process is repeated.

Resource collection and deposition correspond to training the network and evaluating the robot's network, respectively.

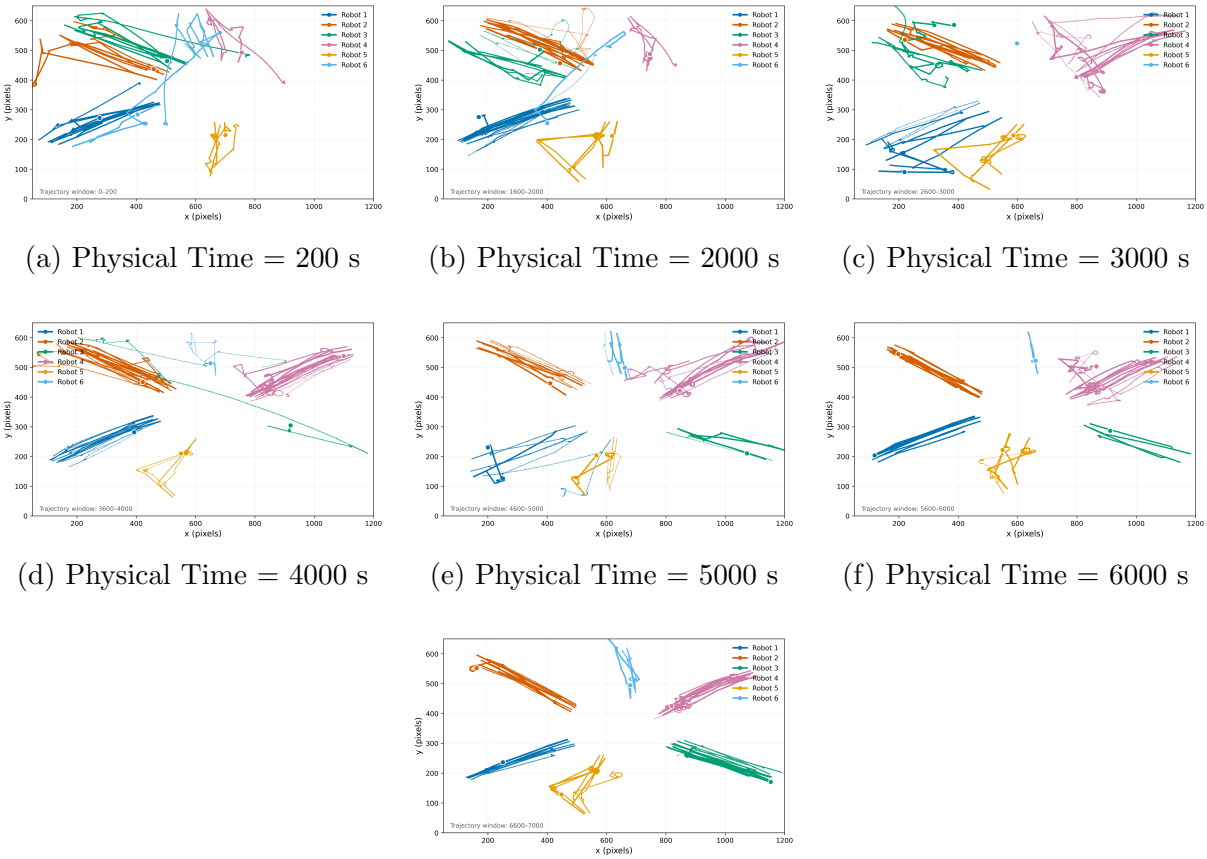
The resulting implementation provides the interface between the stochastic learning model and the physical multi-robot system. The event model determines which agent performs a learning or suppression event, while executing a training event produces the observations and performance changes fed back into the model. This feedback closes the loop between event dynamics and the robots: changes in task performance affect subsequent event rates, while events themselves change the agents' learned task performance.

4.2.5 Results for an Ensemble of Six Robots and Six Tasks

Experiments were carried out with six robots and six resource tasks. Each robot had a neural network with six neurons in the input and output layers, and three hidden layers with 256, 256, and 128 neurons, respectively. The activation function was ReLU, with a dropout rate of 0.2. The robots were trained with a learning rate of 10^{-4}, allowing the networks to learn resource features quickly but increasing simulation runtime. All robots are initialised randomly near the centre such that the robots are facing the blue cylinder and are ready to receive the resources for foraging.

The hyperparameters were chosen such that the maximum coverage fraction is based on the phase diagram shown in Fig. 4.1.2. The experiment was run with $\alpha = 1000$, $\eta = 1000$, and a dead time of 1.5. We chose the step size for tau leaping to allow as many events as possible for multiple robots. For large time steps, multiple robots can get foraging events scheduled; however, this updates the global accuracy matrix less frequently, so the accuracy matrix will change significantly between tau steps, which breaks the assumption of tau leaping [15]. For very small time steps, only a small subset of robots moves at once, which preserves the tau-leaping assumption but increases the overall duration of the experiment. A time step of 0.7 was therefore chosen.

Fig. 4.2.1 and Fig. 4.2.2 show results from one exemplary run with the above hyperparameters. Figures 4.2.1a to 4.2.1f track the robots' trajectories. They start near the centre, just outside the white glossy circle, and terminate at a safe distance from the markers. Because of the central cylinder, the trajectories never cross through the centre; however, they do pass close to the cylinder when they want to approach targets exactly on the



(a) Physical Time = 200 s (b) Physical Time = 2000 s (c) Physical Time = 3000 s

(d) Physical Time = 4000 s (e) Physical Time = 5000 s (f) Physical Time = 6000 s

(g) Physical Time = 7000 s

Figure 4.2.1: Robot trajectories at different stages of the experiment. Each panel shows the trajectory over the preceding 400 frames, with the most recent part shown more prominently. The video was recorded at roughly 30 frames per second.

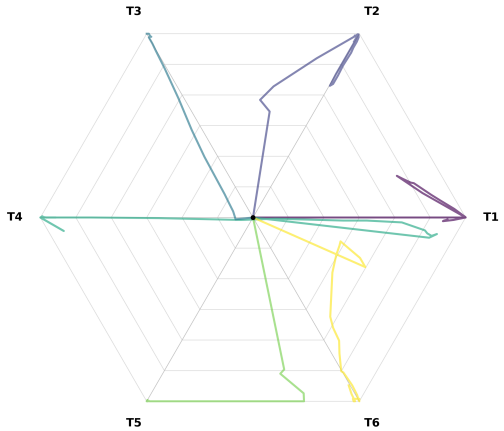


Figure 4.2.2: Barycentric representation of the six robots' accuracies across the six tasks.

opposite side of the cylinder. The robot trajectories can be seen to be moving towards a certain marker then turning almost perpendicularly and going towards a diffffernt marker, something which is clearly seen for robot 5 in plot 4.2.1g. This is because of marker finder fallback which redirects the robot towards the closest marker if it can't find the original marker introduced in earlier sections.

Figures 4.2.1a to 4.2.1f clearly show the emergence of specialisation; robots initally try to forage the same resource; however, the suppression mechanism soon kicks in and breaks the symmetry, with different robots going to different taks and it remains stable with time. Even Figure 4.2.2 showcases this specialisation with directed accuracy values for different robots. The radar plot shows very directed, fast-moving trajectory because of the high learning rate of the simulation.

Chapter 5

Active-Matter Model of Collective Specialisation

In this chapter, we introduce an active matter model that coarse-grains the learning dynamics of neural networks into dynamical and spatial degrees of freedom. Markov networks and state transitions are used to describe the collective dynamics of the ensemble.

5.1 Limitations of the Neural-Agent Ensemble

The neural-ensemble model shows that local performance-dependent interactions can produce emergent specialisation. However, using neural networks as the microscopic dynamical units introduces several limitations when studying the underlying collective dynamics.

A first limitation is the task environment. The numerical experiments in Section 4.1 use synthetic classification tasks designed to produce cross-task interference. This provides a controlled setting in which task competition and specialisation can be studied. Real learning problems, however, can involve correlated data that produce positive transfer or more complex performance landscapes. It is therefore useful to formulate the collective mechanism independently of a particular neural network architecture or dataset construction.

A second limitation is computational. In the neural ensemble, each training event requires optimising the corresponding network, followed by re-evaluating the task-performance variables. Reiterating this procedure for many interacting agents over long stochastic trajectories is computationally expensive, especially as system size and network architecture grow. This major bottleneck limits accessible system sizes, simulation timescales, and the parameter sweeps needed to characterise learning regimes and transitions.

More fundamentally, the neural-network description provides limited insight into the dynamics of the performance variables. The accuracy a_i^k of agent i on task k is an observable obtained from the trained network rather than a variable with a simple physics-based description. This makes it difficult to identify the dynamical mechanisms and competing

timescales responsible for the collective behaviour.

To test the competitive training mechanism independently of these constraints, we develop a model that provides direct control over active improvement, performance degradation, stochastic fluctuations, and interaction-mediated switching. To study the collective dynamics over longer timescales, we introduce the active matter model below.

5.2 Introduction to an Active-Matter Description

Run-and-tumble particles provide a particularly simple case: a particle moves persistently in one direction until a stochastic internal-state transition redirects its propulsion [23]. The mathematical description introduced in Section 2.3 can also be used to construct a coarse-grained description of learning. Instead of interpreting the particle coordinates as positions in physical space, we interpret them as coordinates in *accuracy space*. For an ensemble containing N agents with positions representing task accuracies

$$\mathbf{X}_i(t) = \left(X_i^1(t), X_i^2(t), \ldots, X_i^T(t)\right), \tag{5.2.1}$$

where X_i^k represents the accuracy of agent i on task k with T being the number of tasks. Thus, $\mathbf{X}_i(t)$ is a T-dimensional vector, with each $X_i^k(t)$ representing the accuracy on task k, where accuracy is defined by Eq. 2.1.5. The physically relevant region is

$$0 \leq X_i^k \leq 1. \tag{5.2.2}$$

The task currently being trained corresponds to the direction of motion in the accuracy space and is represented by a discrete internal state

$$M_i(t) \in \{0, 1, \ldots, T\}. \tag{5.2.3}$$

A state $M_i = k$ indicates that particle i is active along task direction k, while $M_i = 0$ denotes an unspecialised state in which the particle is inactive. The active task index therefore replaces the orientation variable of a run-and-tumble particle. The internal state specifies the direction of active propulsion in the T-dimensional accuracy space, and a change of state redirects the active dynamics to another accuracy coordinate. This mapping can be written as:

$$\text{training on task } k \quad \longleftrightarrow \quad \text{active propulsion along } X^k. \tag{5.2.4}$$

Similarly, performance loss on tasks that are not currently being trained is represented by a drift toward lower values of the corresponding coordinates.

This mapping is not intended to establish a one-to-one correspondence between neural-network optimisation and spatial motion. It is instead a phenomenological coarse-graining of the observed performance dynamics, in which low-dimensional stochastic dynamics replace the high-dimensional evolution of the network parameters.

5.3 Model Definition and Numerical Implementation

5.3.1 Performance-Space Dynamics

Consider N particles evolving in a T-dimensional accuracy space, each with a Markov network containing T discrete task states. The coordinate X_i^k represents the position of particle i along task direction k and therefore its performance on task k. Its continuous dynamics are described by the stochastic differential equation

$$dX_i^k = F_i^k(\mathbf{X}, M_i)\, dt + \sqrt{2D}\, dW_i^k, \tag{5.3.1}$$

where D is the noise strength and W_i^k are independent Wiener processes. The deterministic force is given by

$$F_i^k = F_{\text{drive},i}^k + F_{\text{spring},i}^k + F_{\text{wall},i}^k. \tag{5.3.2}$$

The first term describes the active-learning dynamics. When particle i is in internal state $M_i = k$, it experiences a positive propulsion v_0 along that task direction, representing learning in the neural network. Coordinates associated with inactive tasks experience a negative drift with magnitude γv_0. The driving force is therefore

$$F_{\text{drive},i}^k(M_i) = \begin{cases} v_0, & M_i = k, \\ -\gamma v_0, & M_i \neq k, \\ 0 & M_i = 0 \end{cases} \tag{5.3.3}$$

Here v_0 sets the characteristic scale of active improvement and is analogous to the learning rate in neural ensembles. At the same time, γ determines the relative strength of performance degradation across the inactive tasks due to cross-task interference. A higher γ causes faster degradation of inactive tasks, implying high cross-task interference (orthogonal tasks); meanwhile, a lower value leads to slower or no degradation of inactive tasks corresponding to low cross-task interference (correlated tasks). The state $M_i = 0$ produces no drift.

A harmonic contribution is included in the net force acting on the particle, given by

$$F_{\text{spring},i}^k = -k_{\text{spring}} X_i^k. \tag{5.3.4}$$

This term introduces continuous relaxation toward the low-performance region and competes with active propulsion. In the absence of noise and boundary forces, an actively driven coordinate reaches the stationary value

$$X_{stationary} = \frac{v_0}{k_{\text{spring}}}. \tag{5.3.5}$$

In the absence of the harmonic potential, persistent active propulsion causes the particle coordinates to increase approximately linearly with time. This sustained linear improvement does not reflect neural-network learning; hence, the harmonic potential is motivated phenomenologically by neural-network training. For example, compare the linear vs.

asymptotic growth in accuracy in the two plots Fig. 5.4.6a and Fig. 5.4.6b. In simulations, we never operate in the regime $k_{\text{spring}} \gg v_0$ to prevent the spring force from restricting the particle to travel away from the origin.

Since the coordinates represent accuracy, values outside the interval $[0, 1]$ have no physical interpretation. Soft confining forces are therefore applied when a coordinate leaves this interval:

$$F_{\text{wall}}(X) = \begin{cases} -k_{\text{wall}} X, & X < 0, \\ 0, & 0 \le X \le 1, \\ k_{\text{wall}}(1 - X), & X > 1. \end{cases} \tag{5.3.6}$$

The confinement is soft rather than reflecting, so stochastic fluctuations may temporarily move a coordinate outside the interval before the wall force drives it back into the performance domain. Soft confinement is used instead of reflecting boundaries or coordinate clipping to preserve the continuity of the Langevin trajectories and avoid discontinuities in the particle dynamics that could alter the stochastic probability fluxes.

Combining the three contributions gives

$$dX_i^k = \left[F_{\text{drive},i}^k(M_i) - k_{\text{spring}} X_i^k + F_{\text{wall}}(X_i^k)\right] dt + \sqrt{2D}\, dW_i^k. \tag{5.3.7}$$

The particle trajectory is therefore determined by three mechanisms: active motion along the selected task, relaxation, and stochastic exploration of the performance space.

5.3.2 Internal-State Network

Each particle has an internal state, Eq. 5.2.3, that determines its active task direction. The task states $1, \ldots, T$ form the active nodes of the internal Markov network, while state 0 represents the unspecialised state. Fig. 5.3.1 shows the internal-state network. The task states form a fully connected network, allowing a particle to switch from its current task to any other available task. The central state 0 is used when particles are initialised before training and when all task states are temporarily unavailable due to suppression. A particle in this state has no active propulsion and evolves through harmonic relaxation, wall forces, and stochastic diffusion until at least one task becomes available, after which it leaves the central state.

Stochastic transitions between states determine when the direction of active propulsion changes. A transition $S_k \to S_l$ therefore redirects the active force from coordinate X_i^k to X_i^l. The internal Markov dynamics determine the changes in propulsion direction, while the particle position in performance space determines the transition rates.

Two processes contribute to the internal-state dynamics: training events and interaction-induced suppression. Training occurs at a constant rate k_{prod}, representing the rate of learning updates. At each training event, the particle randomly selects one of the currently accessible tasks and updates its internal state. Active propulsion then follows the

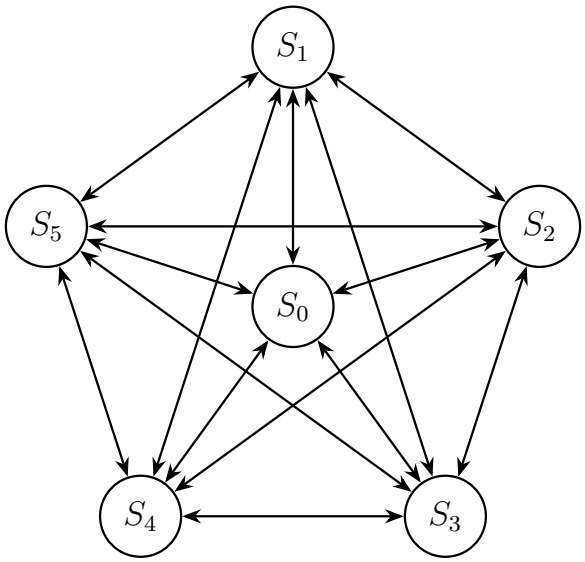


Figure 5.3.1: Schematic representation of the internal-state network of a particle for $T = 5$ tasks. States $S_1, \ldots, S_5$ correspond to the different task states, with transitions between them representing stochastic redirection of the particle's active propulsion from one task coordinate to another. The central state S_0 denotes the unspecialised state, occupied when no task is available. Once a task becomes available and the particle is scheduled for training, the particle leaves S_0 and resumes active motion in the corresponding task direction.

corresponding task-performance coordinate until the next stochastic event.

The second contribution couples the internal dynamics to the particles' positions in accuracy space. The pairwise interaction matches the suppression mechanism of the neural-agent ensemble in Eq.4.1.7. For two particle i and j associated with task k, the rate at which particle i suppresses j is

$$\Gamma^k_{i\leftarrow j}(\mathbf{X}) = \alpha X^k_i X^k_j \sigma\left[\eta\left(X^k_j - X^k_i\right)\right]. \tag{5.3.8}$$

Since particle i may compete with several particles for a task, each pairwise interaction provides a possible suppression channel. The net suppression rate for particle i on task k is obtained by summing over these pairwise channels,

$$\Gamma^k_i(\mathbf{X}) = \sum_{j\neq i} \Gamma^k_{i\leftarrow j}(\mathbf{X}). \tag{5.3.9}$$

As discussed in Section 4.1.4, α is the suppression strength and η controls the sensitivity of suppression to performance differences. The factor $X^k_i X^k_j$ reduces the suppression rate when either particle has low performance on the task. The sigmoid introduces an asymmetry in performance, making suppression more likely when one particle performs better than the other. The interaction therefore acts through the stochastic internal dynamics rather than through a direct mechanical force between particles.

For an active particle that is not in the central unspecialised state, the total instantaneous event rate is

$$\Gamma^k_{i,\mathrm{tot}} = \Gamma^k_i + k_{\mathrm{prod}}. \tag{5.3.10}$$

In the thinning scheme, the probability of a suppression event is

$$P_{\mathrm{sup}} = \frac{\Gamma^k_i}{\Gamma^k_i + k_{\mathrm{prod}}}, \tag{5.3.11}$$

and the probability of a training event is

$$P_{\mathrm{train}} = \frac{k_{\mathrm{prod}}}{\Gamma^k_i + k_{\mathrm{prod}}}. \tag{5.3.12}$$

A suppression event renders the suppressed task temporarily inaccessible for a fixed lock-out time τ. If other tasks remain accessible, the suppressed task is excluded from the accessible set. This temporary suppression alters the network topology by changing the number of accessible states. Subsequently, at the next training event, the particle resumes training by selecting one of the remaining accessible tasks. If no task is available, the particle enters the unspecialised state $M_i = 0$ until at least one task becomes accessible again.

As in the Pfaffenzeller model, suppression affects performance indirectly by changing the tasks available for training and therefore the distribution of training events. Role specialisation emerges from the interaction between stochastic task exploration and these interaction-induced restrictions, paralleling the learning dynamics of the neural-network ensemble.

5.3.3 Numerical Integration of the Continuous Dynamics

We integrate Equation 5.3.7 using the Euler-Maruyama method [43]. For a timestep Δt, the update of each performance coordinate is

$$X^k_i(t + \Delta t) = X^k_i(t) + F^k_i(t)\Delta t + \sqrt{2D\Delta t}\,\zeta^k_i, \tag{5.3.13}$$

where

$$\zeta^k_i \sim \mathcal{N}(0, 1) \tag{5.3.14}$$

are independent standard Gaussian random variables. The stochastic increment $\sqrt{2D\Delta t}\,\zeta^k_i$ is the discrete Euler-Maruyama representation of the Wiener contribution in Eq. 5.3.7. Since the system is actively driven and out of equilibrium, the fluctuation-dissipation relation is not assumed to hold.

5.3.4 State-Dependent Switching by Approximate Thinning

The suppression rate in Eq. 5.3.9 depends explicitly on the continuously evolving particle coordinates $\mathbf{X}(t)$. The standard Gillespie algorithm assumes that the state and propensities remain fixed between successive jump events, so it cannot be applied directly here.

We therefore use the thinning procedure introduced in Section 2.2.4 to sample the state-dependent events [14].

We introduce a constant transition rate $\bar{\Gamma}$ representing the maximum total transition rate within the boundary conditions $[0, 1]$. The sum of the maximum suppression rate and the constant training rate gives

$$\bar{\Gamma} = \alpha(N-1) + k_{\text{prod}}. \tag{5.3.15}$$

The maximum suppression rate is bounded by the sum of the suppression strengths α over the $N-1$ interactions, since both $\sigma(z)$ and $X_i^k X_j^k$ are at most 1. Candidate waiting times are then sampled from the homogeneous Poisson process

$$\Delta t_{\text{cand}} = -\frac{\ln r}{\bar{\Gamma}}, \qquad r \sim \mathcal{U}(0,1). \tag{5.3.16}$$

When the simulation reaches the candidate waiting time, we evaluate the instantaneous suppression rates $\Gamma_i^k(\mathbf{X}(t))$ using the current system configuration, giving the true transition rate Γ_{true}. An event is accepted with probability

$$P_{\text{acc}} = \frac{\Gamma_{true}}{\bar{\Gamma}}. \tag{5.3.17}$$

If the event is rejected, no internal-state transition occurs and a new candidate waiting time is generated. If the event is accepted, it is classified as either suppression or training according to Eq. 5.3.11 and Eq. 5.3.12. This follows the standard thinning procedure: candidate events are generated from a dominating Poisson process and accepted according to their ratio to the dominating rate [14].

Large stochastic fluctuations can temporarily move particle positions outside $[0, 1]$. Therefore, Eq. 5.3.15 is an operational bound within the simulated parameter regime rather than an unconditional mathematical bound over $\mathbf{X} \in \mathbb{R}^{N,T}$.

5.3.5 Time-Discretised Event Evaluation

The thinning procedure is coupled to the finite-timestep integration of the Langevin dynamics. Candidate jump times are generated continuously according to Eq. 5.3.16; however, the simulation loop checks whether the candidate waiting time to an event has been reached only at discrete times $t_n = n\Delta t$. The resulting algorithm is therefore an approximation to an exact continuous-time thinning procedure. Its accuracy requires a small timestep; additionally, it should also be small compared to both the characteristic timescales of continuous dynamics and the timescale of state-dependent transitions. We can represent this as

$$\Delta t \ll \min\left(t_{\text{motion}}, t_{\text{switch}}, t_\Gamma, t_\gamma\right), \tag{5.3.18}$$

Here, t_{motion} describes the evolution of the continuous coordinates and diffusive fluctuations, t_{switch} is the typical interval between stochastic events, t_Γ is the timescale over which $\Gamma_i^k(\mathbf{X}(t))$ changes significantly, and t_γ is the typical timescale of γ-driven cross-task interference. As $\Delta t \to 0$, the delay between a candidate event time and its numerical evaluation decreases, so the finite-timestep implementation approaches the intended continuously evolving switching process.

5.3.6 Simulation Algorithm

The complete numerical dynamics combines the evolution of the discrete internal states with continuous updates of the performance coordinates. At each integration step, we first determine the available tasks from the lockout times and return tasks whose lockout periods have ended to the accessible set. For each particle, we calculate the instantaneous suppression rate from Eq. 5.3.9. The candidate event is then accepted or rejected according to Eq. 5.3.17, and accepted events are classified as suppression or training. A training event selects an accessible task direction. A suppression event locks the current task for time τ and sets the particle to state zero. After the internal state is updated, the forces are evaluated for every particle and task coordinate according to Eq. 5.3.13. The resulting particle configuration then determines the suppression rates at later candidate events.

5.4 Collective Behaviour and Reproduction of Neural-Ensemble Phenomena

We developed the active-matter model to test whether the collective phenomena observed in the neural ensemble can arise from a smaller set of non-equilibrium dynamical ingredients. The following sections present the results of the active-matter model and compare them with the neural system.

5.4.1 Emergence of Stable Specialisation

The first test of the active-matter model is whether symmetry breaking among identical particles produces specialised states, as observed in the neural ensemble. The particle positions are initialised from a narrow low-performance interval,

$$X_i^k(0) \sim \mathcal{U}(0.01, 0.1), \tag{5.4.1}$$

and internal state $M_i = 0$. Small performance differences produced by fluctuations and initial positional asymmetries are amplified by the sigmoidal interaction in Eq. 5.3.9, which preferentially suppresses the lower-performing particle. The suppressed particle then transitions to state 0, while the other particles continue along their current directions. This creates a feedback mechanism. Without sufficiently strong suppression, particles continue to sample the available task states and do not form stable specialisation, as in the neural ensemble model. Interaction-induced suppression removes competing tasks from the accessible set, allowing particles to spend longer periods on a subset of tasks. The resulting performance differences are then amplified by the continuous Langevin dynamics, producing stable specialisation.

An alternative modelling choice is to redirect a particle to one of the available tasks when it is suppressed, rather than sending it to state zero. This allows the particle to retain active propulsion instead of waiting for the next training event.
Fig. 5.4.1 shows the specialised configuration of the system. Specialisation is characterised not only by the magnitude of individual coordinates but also by the distribution

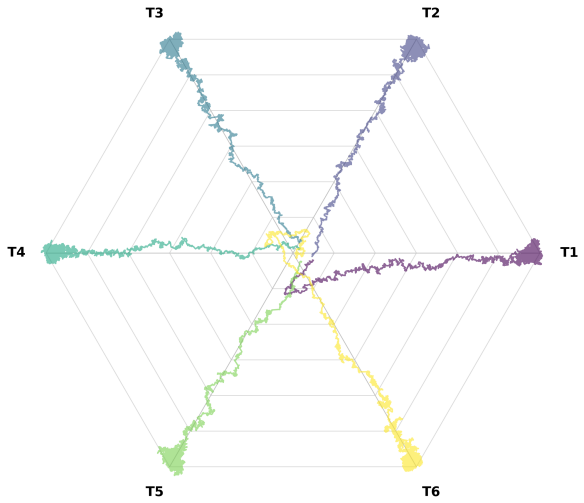


Figure 5.4.1: Evolution of the six particle ($N = 6$) trajectories in the six-dimensional performance space ($T = 6$), visualised as radar-plot trajectories. Each coloured trajectory corresponds to one particle and shows the time evolution of its task-accuracy coordinates from the initially unspecialised state to a specialised configuration. The trajectories converge towards distinct task axes, demonstrating the emergence of role specialisation driven by repeated stochastic training on accessible tasks together with interaction-induced suppression. Simulation parameters: $v_0 = 0.1$, $\gamma = 0.09$, $D = 9 \times 10^{-5}$, $k_{\text{spring}} = 0.09$, $\alpha = 800$, $\eta = 30$, $\tau = 6$, $k_{\text{prod}} = 1.0$, $k_{\text{wall}} = 1000$, $\Delta t = 0.01$, $T_{\text{sim}} = 100$, and seed 42.

of active directions across the ensemble. A specialised state corresponds to different particles becoming localised along different task axes rather than all particles increasing all task coordinates simultaneously.

5.4.2 Collective Phase Diagram

The competition between persistent training and interaction-mediated suppression produces different dynamical regimes. As for the neural ensemble phase diagram in Figure 4.1.2, a phase diagram can be constructed for the active-matter model, shown in Figure 5.4.2.

Although both phase diagrams show a transition from an unspecialised regime to one with near-complete task coverage, their phase-boundary geometries differ. The neural ensemble shows a sharper transition with a more clearly defined threshold, whereas the active-matter model has a broader transition region without a sharp phase boundary. This difference follows from the different representations of the learning dynamics. In the neural ensemble, discrete training and suppression events directly change neural-network performance. In the active-matter model, the performance coordinates evolve continuously through active propulsion, inactive-task decay, and diffusion. Performance differences therefore accumulate between suppression events. The active-matter model

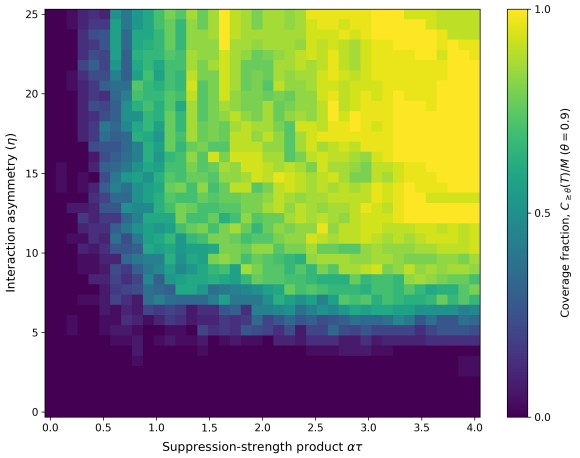


Figure 5.4.2: Phase diagram of the active-matter model showing the steady-state task coverage fraction, $C_\theta(T)/N$, with threshold $\theta = 0.9$, as a function of the suppression-strength product $\alpha\tau$ and the interaction asymmetry parameter η. Each point represents the average over five independent simulations. Dark colours indicate poor task coverage, meaning only a few tasks are occupied by specialised particles. In contrast, bright colours indicate complete task coverage, with specialised particles representing all six tasks simultaneously. Simulations were performed with $N = 6$ particles, $T = 6$ tasks, $v_0 = 0.1$, $\gamma = 0.09$, $D = 5 \times 10^{-4}$, $k_{\text{spring}} = 0.09$, $k_{\text{wall}} = 1000$, $k_{\text{prod}} = 1.0$, $\Delta t = 0.02$, and a total simulation time of $T_{\text{sim}} = 60$.

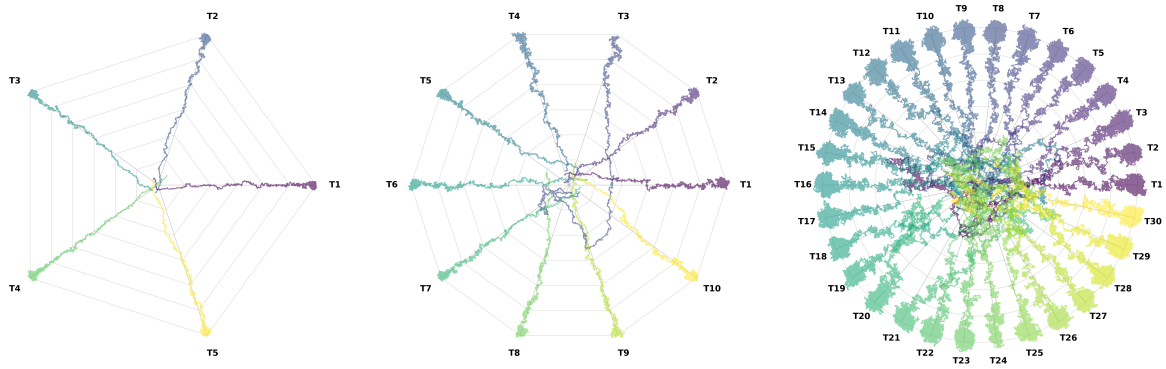
consequently develops specialisation progressively as interaction-induced restrictions on the accessible task set bias the stochastic training dynamics toward a smaller subset of tasks. The continuous dynamics then amplify these differences into stable specialisation.

5.4.3 Scaling with System Size

We also examine whether collective task coverage persists as the numbers of particles and tasks change. The effective interaction experienced by a particle depends on the system size. Increasing N increases the number of potential competitors, while increasing T increases the number of task states over which particles can specialise.
The scaling behaviour in Fig. 5.4.3 shows that specialisation persists across the tested system sizes in the active-matter model, as also observed in the neural ensemble model.

5.4.4 Plasticity and Memory

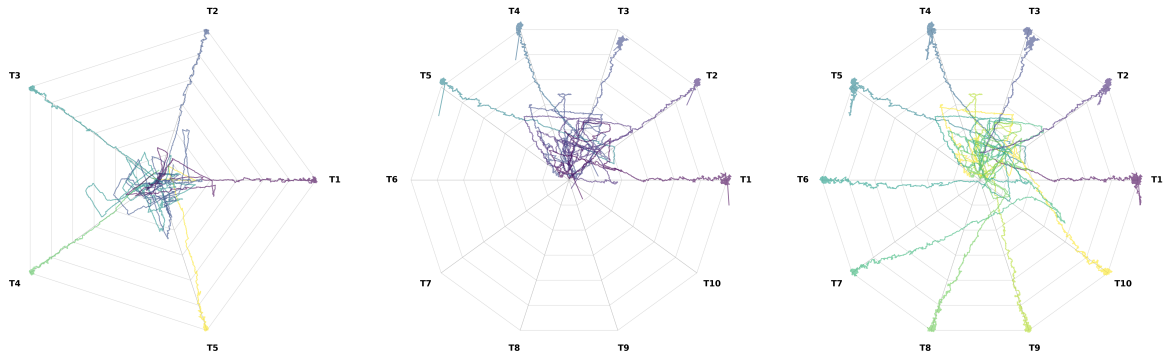
The neural ensemble model was developed as a multi-agent solution to the plasticity-and-memory dilemma. As shown in Figures 4.1.4 and 4.1.5, the neural ensemble demonstrated that the system is plastic enough to reconfigure in response to perturbations while retaining memory at the individual level. Experiments similar to those of neural ensembles can be recreated using the active-matter model.



(a) $N = 5$ and $T = 5$ (b) $N = 10$ and $T = 10$ (c) $N = 30$ and $T = 30$

Figure 5.4.3: Stable Specialisation across system size

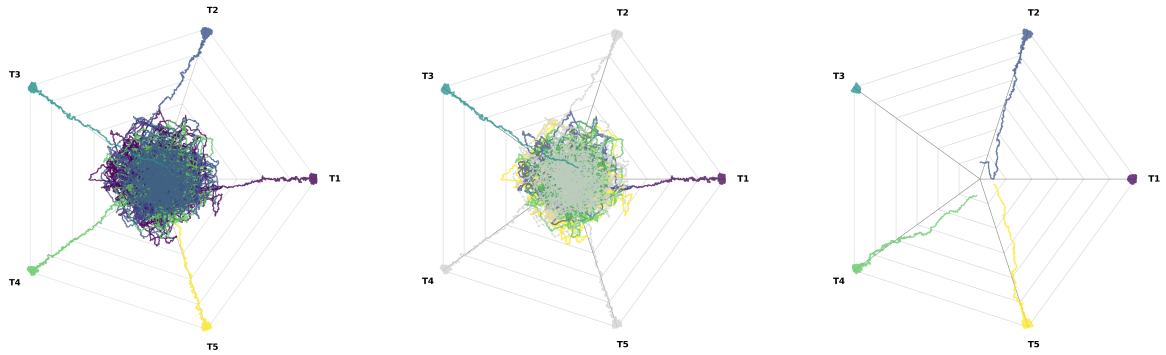
To reproduce the perturbation in Figure 4.1.4, the simulation was initialised with $N = 10$ and $T = 5$. Stable specialisation emerged with 5 particles localised along the 5 task directions, while the remaining particles remained unspecialised near the origin. We then added 5 tasks. Particles near the origin became active on the new tasks, and the system reorganised into a new set of specialised roles. Figure 5.4.4 summarises the results.



(a) Before Perturbation ($N = 10$ and $T = 5$) (b) Perturbation Applied ($N = 10$ and $T = 10$) (c) After Perturbation ($N = 10$ and $T = 10$)

Figure 5.4.4: Response of the active-matter model to adding new tasks. (a) Steady-state specialisation before the perturbation with $N = 10$ particles and $T = 5$ tasks. (b) Immediately after introducing five additional tasks, increasing the task space from $T = 5$ to $T = 10$. (c) Final steady state, showing the emergence of a new specialisation pattern across all $T = 10$ tasks. The figure demonstrates the active-matter model's ability to adapt to an expanded task environment while preserving collective specialisation through reorganisation of the particle population.

Similarly, the perturbation in Figure 4.1.5 can be reproduced by randomly removing half of the particles from a specialised configuration. The remaining unspecialised particles can then explore new configurations and reorganise the system. The results are shown in Figure 5.4.6.



(a) Before Perturbation ($N = 10$ and $T = 5$)

(b) Perturbation Applied ($N = 5$ and $T = 5$)

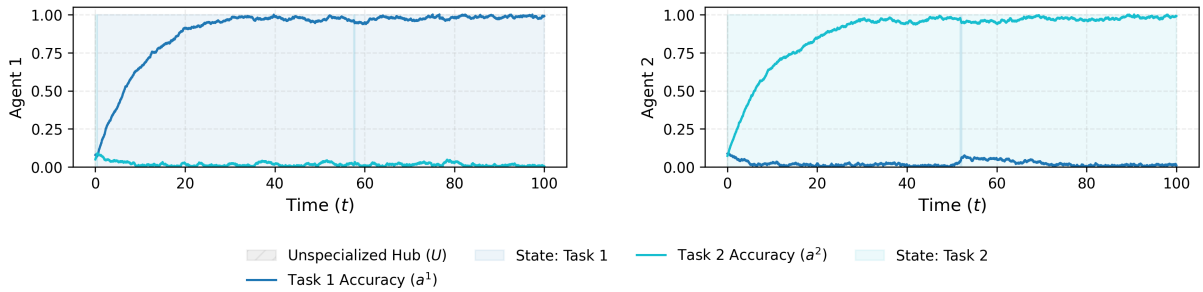
(c) After Perturbation ($N = 5$ and $T = 5$)

Figure 5.4.5: Response of the active-matter model to particle removal. (a) Steady-state specialisation before the perturbation with $N = 10$ particles and $T = 5$ tasks. (b) Perturbation in which five particles are randomly removed from the system, reducing the population to $N = 5$ while keeping the task space unchanged. (c) Final steady state, showing the reorganisation of the remaining particles to maintain task coverage and collective specialisation. The figure demonstrates the robustness of the specialisation dynamics against the loss of a significant fraction of the particle population.

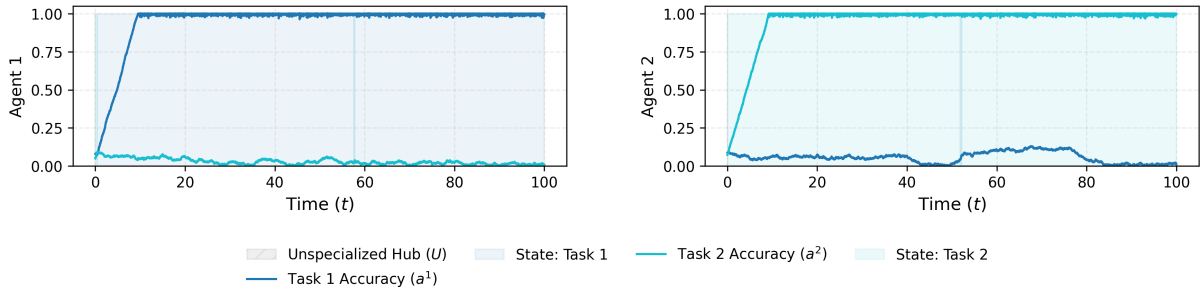
5.4.5 Role of Harmonic Relaxation

The parameter k_{spring} controls the tendency of the continuous coordinates to relax toward the origin and acts as a coarse-grained representation of resistance to learning, or effective task difficulty. It therefore competes directly with the active propulsion v_0. The balance between propulsion and relaxation determines the increase in the accuracy coordinate before stochastic events redirect the particle to another accessible task.

Unlike a model with only active propulsion, harmonic relaxation prevents the performance coordinates from increasing indefinitely. Without this term, persistent training produces an approximately linear increase in performance with time, as shown in Figure 5.4.6a. This differs from the learning dynamics of neural networks, where performance generally increases rapidly at early stages before approaching a slower rate of improvement [7]. The relaxation strength therefore provides a coarse-grained representation of *task difficulty*. Smaller values of k_{spring} allow improvements to persist more readily and lead to higher performance under sustained training, while larger values make performance more difficult to increase and more readily lost when training is interrupted. This provides a way to study different task-difficulty regimes.



(a) $k_{\text{spring}} = 0.1$



(b) $k_{\text{spring}} = 0.001$

Figure 5.4.6: Effect of the harmonic relaxation strength k_{spring} on the evolution of task-performance coordinates for a system with $N = 2$ particles and $T = 2$ tasks. (a) For a relaxation strength of $k_{\text{spring}} = 0.1$. (b) For a relaxation strength of $k_{\text{spring}} = 0.001$. The ratio of the harmonic relaxation and propulsion controls both the characteristic accuracy scale and prevents the unbounded linear growth that would arise from persistent active propulsion alone.

5.4.6 Role of Diffusive Fluctuations

The diffusion coefficient D controls the amplitude of stochastic fluctuations in performance space. Noise has two effects in the model. Weak fluctuations provide the initial asymmetries from which otherwise equivalent particles can begin to differentiate. Once a specialised configuration has formed, stronger diffusion can disrupt this localisation by moving particles away from the trajectories generated by their active states. The dependence on D therefore measures the stability of the specialised state under stochastic perturbations.

5.4.7 Role of Inactive-Task Decay and Task Orthogonality

The parameter γ controls the rate at which performance on tasks that are not currently being trained decays relative to the active learning rate v_0. The active task experiences positive propulsion of magnitude v_0, while inactive task coordinates drift by $-\gamma v_0$. Thus, γ determines the strength of cross-task interference and provides a coarse-grained analogue of task orthogonality in the neural-agent ensemble.

Small values of γ correspond to weak interference between tasks, allowing performance gains on previously trained tasks to persist for longer. Larger values correspond to stronger

task interference, causing inactive performance coordinates to decay more rapidly unless future training events revisit them. Rather than explicitly modelling correlations between datasets or neural-network representations, the inactive-task decay rate provides a coarse-grained representation of task overlap. The competition between active learning and inactive-task decay introduces a characteristic forgetting timescale

$$t_{\text{forget}} \sim \frac{1}{\gamma v_0}. \tag{5.4.2}$$

This determines how long performance on an inactive task persists without further training. Consequently, γ plays a central role in controlling both the stability of specialised states and the balance between memory and plasticity.

5.5 Summary, Comparison, and Outlook

In the active-matter model, neural-network training is coarse-grained as continuous motion in accuracy space, while the global Gillespie process of the neural ensemble is represented by particle-wise thinning processes for stochastic transitions between Markov-network states. The model allows the specialisation mechanism to be studied under different task-difficulty and task-orthogonality conditions, with stochastic perturbations and over longer timescales. The construction provides a lower-dimensional nonequilibrium model in which the collective specialisation observed in the neural ensemble can be examined.

The two models organise their stochastic dynamics differently. The neural ensemble evolves through a single Gillespie process that samples interactions across the population, whereas the active-matter model assigns each particle an independent state-dependent thinning process that governs transitions between its internal task states. Section 5.4.2 discusses the consequences of these modelling choices for the collective dynamics.

The active matter model can test the methods in [6] across task complexity and task orthogonality; however, we did not use it to verify whether stable organisation can emerge across this wide range of cases, so this remains an open area of investigation. We believe it would be interesting to study how parameters such as v_0, γ, k_{spring}, D, α, and τ independently control identifiable dynamical mechanisms and how their variation makes it possible to determine which ingredients are sufficient for the emergence of collective specialisation and to identify the dynamical mechanisms responsible for the transition.

Chapter 6

Discusssion

Figures 4.2.1a–4.2.1g and Fig. 4.2.2 show the emergence of specialisation. They also show that the agents initially explore the experimental arena before the suppression dynamics break the symmetry. Initially, robot 6 attempts to forage for resources 1, 2, and 3 and continues to explore different tasks until at least 2000 seconds into the experiment. This activates the suppression mechanism, and robot 3 can be seen to be suppressed and therefore stationary at 3000 s. The suppression ends the robot's exploration phase, after which the robot specialises in foraging for a single resource and continues to do so for the remainder of the experiment.

Robots 2 and 3 initially forage for the same resource, as shown in Figs. 4.2.1a–4.2.1c. Until suppression occurs, the robots explore unoccupied foraging resources and travel across the experimental arena. During this exploration phase, the robot trajectories are generally broader between the resource deposit regions and the corresponding markers because the robots perform collision avoidance. This causes deviations from the direct path and increases the overall spread of the trajectories. From Fig. 4.2.1d to Fig. 4.2.1g, the reduced spread of the trajectories indicates the emergence of specialisation and the end of the exploration phase.

These results show that specialisation can emerge in a robotic system performing reverse central foraging experiments without explicit task assignment. They also show how memory can arise at the system level through local interactions between robots. An open question is whether the specialised roles remain stable under perturbations, such as the addition or removal of resources or robots. The next step is therefore to test whether plasticity can emerge naturally in the system.

The neural ensemble experiments of Pfaffenzeller and the active-matter model suggest that plasticity can arise in the system. This motivates further investigation of the robustness of the specialised configuration and its ability to reorganise under physical constraints.

Both the neural ensemble and the collective of intelligent robots perform classification tasks in supervised learning settings. The neural ensemble performs tasks such as predicting labels across different datasets, while the robots classify resources, with network

accuracy serving as the competitive metric in both cases. It remains to be investigated whether the supervision-driven dynamics can also lead to specialisation when other competitive metrics are used.

Bibliography

[1] Timothée Lesort, Vincenzo Lomonaco, Andrei Stoian, Davide Maltoni, David Filliat, and Natalia Díaz-Rodríguez. Continual learning for robotics: Definition, framework, learning strategies, opportunities and challenges. *Information fusion*, 58:52–68, 2020.

[2] German I Parisi, Ronald Kemker, Jose L Part, Christopher Kanan, and Stefan Wermter. Continual lifelong learning with neural networks: A review. *Neural networks*, 113:54–71, 2019.

[3] Brian P Gerkey and Maja J Matarić. A formal analysis and taxonomy of task allocation in multi-robot systems. *The International journal of robotics research*, 23(9):939–954, 2004.

[4] Levent Bayındır. A review of swarm robotics tasks. *Neurocomputing*, 172:292–321, 2016.

[5] Qi Lu, G Matthew Fricke, John C Ericksen, and Melanie E Moses. Swarm foraging review: Closing the gap between proof and practice. *Current Robotics Reports*, 1(4):215–225, 2020.

[6] Nikolas Pfaffenzeller. Decentralized multi-agent learning and emergent task specialization. Master's thesis, Ludwig Maximilian University of Munich, 2026.

[7] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. `http://www.deeplearningbook.org`.

[8] Abien Fred Agarap. Deep learning using rectified linear units (relu). *arXiv preprint arXiv:1803.08375*, 2018.

[9] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. Learning representations by back-propagating errors. *nature*, 323(6088):533–536, 1986.

[10] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

[11] Nicolaas Godfried Van Kampen. *Stochastic processes in physics and chemistry*, volume 1. Elsevier, 1992.

[12] Crispin Gardiner. *Stochastic methods*, volume 4. Springer Berlin, 2009.

[13] Daniel T Gillespie. Exact stochastic simulation of coupled chemical reactions. *The Journal of Physical Chemistry*, 81(25):2340–2361, 1977.

[14] PA W Lewis and Gerald S Shedler. Simulation of nonhomogeneous poisson processes by thinning. *Naval research logistics quarterly*, 26(3):403–413, 1979.

[15] Daniel T Gillespie. Approximate accelerated stochastic simulation of chemically reacting systems. *The Journal of chemical physics*, 115(4):1716–1733, 2001.

[16] Ron Solan and Gad Getz. An exact tau-leaping method. *arXiv preprint arXiv:2509.12631*, 2025.

[17] Jürgen Schnakenberg. Network theory of microscopic and macroscopic behavior of master equation systems. *Reviews of Modern physics*, 48(4):571, 1976.

[18] Udo Seifert. Stochastic thermodynamics, fluctuation theorems and molecular machines. *Reports on Progress in Physics*, 75(12):126001, 2012.

[19] RKP Zia and B Schmittmann. Making a mountain out of a molehill: a response to "comment on 'probability currents as a roadmap to non-equilibrium steady states'". *Journal of Statistical Mechanics: Theory and Experiment*, 2007(07):P07012, 2007.

[20] Pawel Romanczuk, Markus Bär, Werner Ebeling, Benjamin Lindner, and Lutz Schimansky-Geier. Active brownian particles: From individual to collective stochastic dynamics. *The European Physical Journal Special Topics*, 202(1):1–162, 2012.

[21] Clemens Bechinger, Roberto Di Leonardo, Hartmut Löwen, Charles Reichhardt, Giorgio Volpe, and Giovanni Volpe. Active particles in complex and crowded environments. *Reviews of modern physics*, 88(4):045006, 2016.

[22] Howard C Berg and Douglas A Brown. Chemotaxis in escherichia coli analysed by three-dimensional tracking. *nature*, 239(5374):500–504, 1972.

[23] Mark J Schnitzer. Theory of continuum random walks and application to chemotaxis. *Physical Review E*, 48(4):2553, 1993.

[24] ELEGOO. Elegoo smart robot car kit v4.0 with camera. `https://www.elegoo.com/en-gb/products/elegoo-smart-robot-car-kit-v-4-0`, 2026. Official product documentation and specifications.

[25] ELEGOO. Elegoo smart robot car kit v4.0 new. `https://github.com/elegooofficial/ELEGOO-Smart-Robot-Car-Kit-V4.0-New`, 2026. Official firmware repository, version 2.1.2.

[26] Prusa Research. Prusa-firmware-esp32-cam documentation: Freenove esp32-s3-wroom. `https://github.com/prusa3d/Prusa-Firmware-ESP32-Cam/blob/master/doc/Freenove%20ESP32-S3-Wroom/README.md`, 2023. Accessed: 2026-08-23.

[27] Arduino. Arduino uno rev3 technical specifications. `https://store.arduino.cc/products/arduino-uno-rev3`, 2024. Accessed: 2026-07-30.

[28] ElecFreaks. *HC-SR04 Ultrasonic Ranging Module Data Sheet*. ElecFreaks Technology Co., Ltd. Accessed: 2026-08-24.

[29] Umakanta Nanda and Sushant Kumar Pattnaik. Universal asynchronous receiver and transmitter (uart). In *2016 3rd International Conference on Advanced Computing and Communication Systems (ICACCS)*, volume 01, pages 1–5, 2016.

[30] Paul Horowitz and Winfield Hill. *The Art of Electronics*. Cambridge University Press, 3 edition, 2015.

[31] Wojciech Skut, Jakub Piskorski, and Jan Daciuk. Finite-state machines. 2005.

[32] Ouarda Zedadra, Nicolas Jouandeau, Hamid Seridi, and Giancarlo Fortino. Multi-agent foraging: State-of-the-art and research challenges. *Complex Adaptive Systems Modeling*, 5(3), 2017.

[33] Qi Lu, G. Matthew Fricke, John C. Ericksen, and Melanie E. Moses. Swarm foraging review: Closing the gap between proof and practice. *Current Robotics Reports*, 1:215–225, 2020.

[34] Edwin Olson. Apriltag: A robust and flexible visual fiducial system. In *2011 IEEE international conference on robotics and automation*, pages 3400–3407. IEEE, 2011.

[35] ELEGOO. Elegoo smart robot car kit v4.0 user manual. `https://www.elegoo.com/blogs/arduino-projects/elegoo-smart-robot-car-kit-v4-0-tutorial`, 2023. Accessed: 2026-07-30.

[36] Bruce D. Lucas and Takeo Kanade. An iterative image registration technique with an application to stereo vision. In *Proceedings of the 7th International Joint Conference on Artificial Intelligence*, volume 2, pages 674–679, 1981.

[37] Carlo Tomasi and Takeo Kanade. Detection and tracking of point features. Technical Report CMU-CS-91-132, Carnegie Mellon University, 1991.

[38] Jianbo Shi and Carlo Tomasi. Good features to track. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 593–600, 1994.

[39] R. L. Jeanne. Evolution of social behavior in the vespidae. *Annual Review of Entomology*, 25:371–396, 1980.

[40] S. Sumner et al. Dynamic reorganization in primitively social wasp colonies. *Proceedings of the Royal Society B*, 273:19–26, 2006.

[41] Solenn Patalano, Adolfo Alsina, Carlos Gregorio-Rodríguez, Martin Bachman, Stephanie Dreier, Irene Hernando-Herraez, Paulin Nana, Shankar Balasubramanian, Seirian Sumner, Wolf Reik, et al. Self-organization of plasticity and specialization in a primitively social insect. *Cell Systems*, 13(9):768–779, 2022.

[42] Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the thirteenth international conference on artificial intelligence and statistics*, pages 249–256. JMLR Workshop and Conference Proceedings, 2010.

[43] Desmond J Higham. An algorithmic introduction to numerical simulation of stochastic differential equations. *SIAM review*, 43(3):525–546, 2001.

Declaration of authorship

I hereby declare that this thesis is my own work, and that I have not used any sources or aids other than those stated in the thesis. AI-based paraphrasing and grammar checking tools were used in the preparation of the thesis. The data and code used for the thesis can be made available upon request.

Munich, 24.09.2026
Location, date

Name